

SS2026\_IBS6\_IA\_GR01

# Demand Management: Forecasting Product Demand Across Global Warehouses

*Full Project Reference Document — complete methodology, analysis & results*

---

Presented by: **May Zin Ko** · **Nashitah**

Course: IBS6 — Industrial Analytics | Final Presentation | SS 2026



*Red badge = slide presented live (15 min, 2 speakers)*

# What This Document Covers

**01**

## **Introduction**

Idea, scope, objectives, tasks, milestones & deliverables

**02**

## **Business Case**

Business understanding, theoretical background, stakeholders

**03**

## **Dataset**

Source, structure, quality, preparation, descriptive analytics

**04**

## **ML Algorithms — May Zin Ko**

ARIMA/ARMA & Random Forest: design, tuning, results

**05**

## **ML Algorithms — Nashitah**

SARIMA, XGBoost & Prophet: design, validation, results

**06**

## **Model Comparison**

All five models compared — warehouse & product level

**07**

## **Business Impact**

Quantifying benefits, operating cost, risks & recommendations

**08**

## **Summary & Lessons Learned**

Project recap, 6 key lessons, recommendations for future work

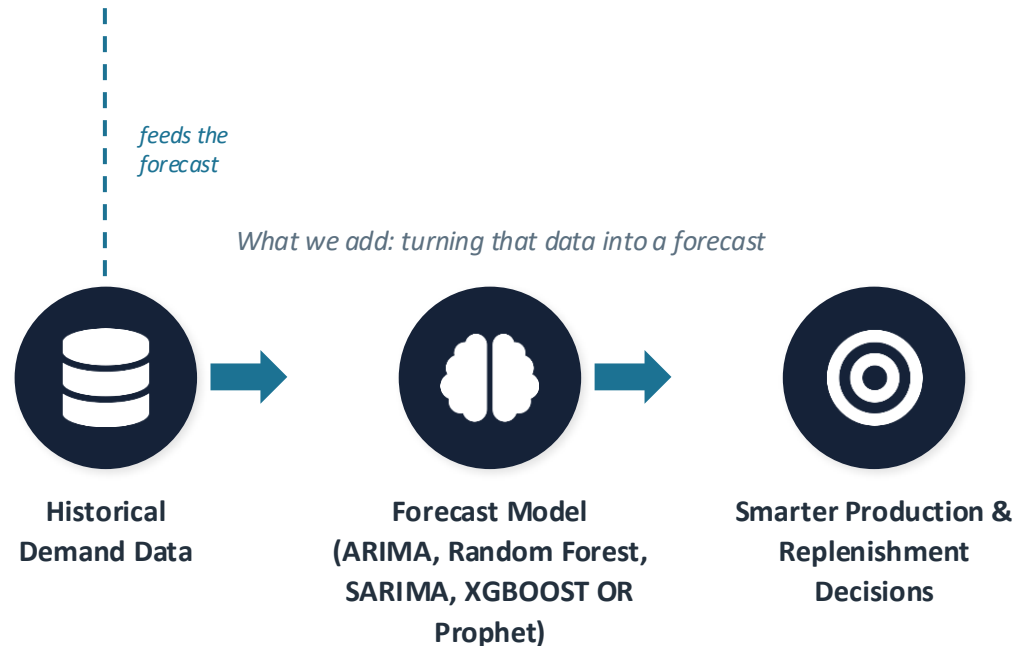
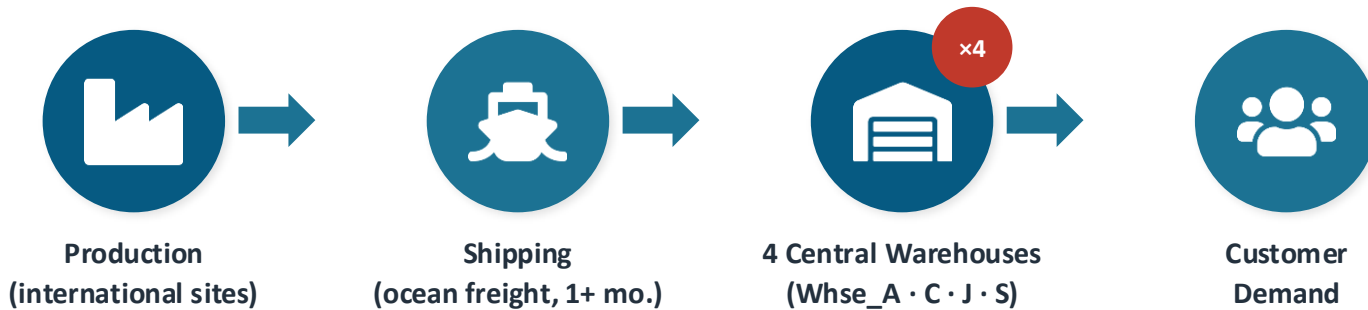
# 01

## Introduction

*Idea · Scope · Objectives · Tasks & Milestones · Deliverables*

# The Business, at a Glance

*Physical flow of goods*



## This Project

We use historical order demand at each warehouse to forecast what's coming next — so production and replenishment decisions can be made before demand arrives, not after.

## The Loop

Better decisions reduce stockouts and excess inventory — which changes future demand patterns, which feed back into the next forecast.

*Continuous improvement loop*

Source: Zhao, F. (n.d.). Product demand forecasting [Dataset]. Kaggle. <https://www.kaggle.com/datasets/felixzhao/productdemandforecasting>

# The Idea — What Are We Solving?

This project analyzes historical product demand data from a global manufacturing company that operates across multiple international production sites and distributes products through four central warehouses, each serving its own region.

## Core Questions We're Answering

- 1 **How many** products should we produce?
- 2 **When, Which products** should we stock them in which warehouse?
- 3 **How** should we store products in different warehouses?

### In One Sentence

*Demand analysis by product across multiple warehouses to support inventory planning.*

---

Over 1 million transactional records covering a wide range of products and order dates give a strong foundation for this analysis.

# Scope of the Study

The analysis covers demand patterns across four central warehouses and multiple product categories over the available time period in the dataset. Findings are based solely on the historical data provided and are intended to support demand planning decisions at the warehouse level.

## In Scope

- Exploratory data analysis — demand behavior, trends, seasonality, irregularities
- Data preprocessing and cleaning to prepare the dataset for modeling
- Building and evaluating forecasting models to predict monthly product demand
- Comparing model results to identify the most suitable approach per warehouse / product

## Out of Scope

- No external factors modeled (pricing, promotions, marketing campaigns)
- No real-time / live data integration — historical batch analysis only
- No SKU-level forecasting beyond the top-4 highest-demand products

# Objectives

1

## **Analyze historical demand patterns**

Understand demand behavior across products, warehouses & time periods

2

## **Identify trends & seasonality**

Detect intermittent demand behavior and irregular patterns

3

## **Develop a forecasting model**

Capable of predicting monthly product demand

4

## **Evaluate model performance**

Using appropriate forecasting accuracy metrics (MAE, RMSE, MAPE)

5

## **Support better replenishment decisions**

Improve inventory management using demand insights

# Evaluation of Success

*Success is measured on both technical performance and practical business usefulness.*

## Technical Criteria

Forecasting accuracy is measured using standard error metrics. A lower error value indicates a more accurate and reliable forecast.

MAE — average size of forecast errors

RMSE — penalizes larger errors more heavily

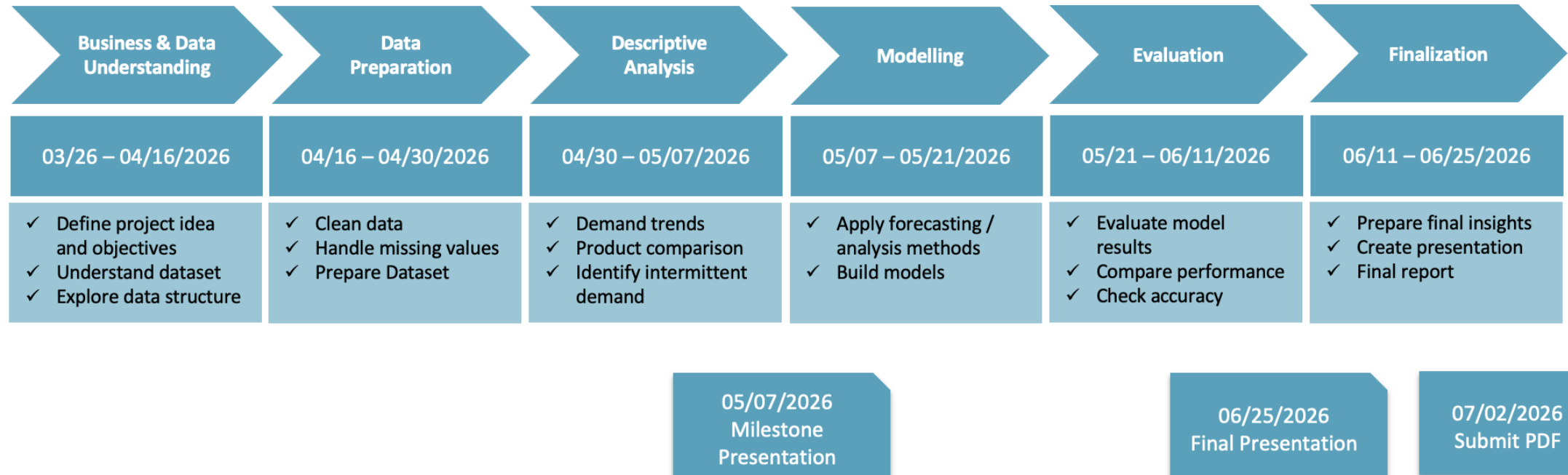
MAPE — scale-free % error for cross-warehouse comparison

## Business Criteria — Considered Successful If:

- Demand patterns and trends are clearly identified
- The forecasting models produce reliable estimates that could realistically support inventory planning decisions
- A meaningful comparison between models is made to identify the best approach for this dataset
- Overview of TOP Demand PRODUCTS in which warehouse we should store them also better if find out which season or months are these demanded mostly. Find out seasonal patterns.



# Tasks & Milestones



● **Where we are now:** Evaluation phase complete (Tasks 1–10 done) — currently in Finalization (Task 11). Nashitah's models (SARIMA, XGBoost, Prophet) are now integrated, with cross-model comparison complete.

# Deliverables

## Clean Dataset

demand\_cleaned.csv — cleaned, typed, deduplicated, ready for modeling

## Demand Analysis Report

Shared group notebook: business case, dataset, EDA, solution design, business impact & lessons learned

## Individual ML Notebooks

May Zin Ko: ARIMA/ARMA & Random Forest.  
Nashitah: SARIMA, XGBoost & Prophet.

## Final Presentation

This document — full reference deck plus the 15-minute live walkthrough

# 02

## Business Case

*Business Understanding · Problem Statement · Theoretical Background · Stakeholders*

# Business Understanding

## Real-World Scenario

- Demand is unpredictable — some products rarely needed, some frequently requested
- Demand varies across different warehouses
- This creates challenges in balancing availability and cost

## The Global Supply Chain Context

The company operates in a highly complex global supply chain environment where demand planning is a critical challenge. Products are manufactured across multiple international locations and must travel long distances via ocean freight before reaching any of the four central warehouses.

**Lead time > 1 month:** procurement and restocking decisions must be committed well before actual customer demand is known.

# The Demand Challenge

*Demand is unpredictable and changes over time across products and warehouses. This creates a trade-off between product availability and inventory cost.*

## Intermittent Products

Rare but important demand spikes — hard to predict, but stockouts on these items still hurt.

## Fast-Moving Products

Require consistent replenishment — predictable, but demand patterns still differ by warehouse, creating uneven stock pressure.



***Without demand visibility, firms risk overstock in one warehouse and stockouts in another — at the same time.***

# Problem Statement

## Overstocking

Leads to high inventory holding costs

## Understocking

Leads to stockouts and operational delays

## Irregular Demand

Intermittent demand makes planning difficult

## No Demand Insight

Results in inefficient replenishment decisions



### The Core Tension

If warehouses stock too little, they risk running out of products before the next shipment arrives. If they stock too much, storage costs increase and warehouse space is wasted. This balance between supply and demand is at the core of the business problem this project addresses.

# Goal & Value Proposition

## Goal

- Improve demand understanding to support better inventory and replenishment planning
- Identify category-wise, product-wise, and warehouse-wise demand patterns
- Support decisions on how much to stock, when to replenish, and where to position inventory
- Build a foundation for demand forecasting and data-driven supply chain planning

## Value Proposition

This project transforms raw historical demand data into meaningful insights that support better business decision-making. Rather than relying on estimates or intuition, the company can use data-driven forecasts to plan inventory more accurately.

---

*By doing so, the company can reduce unnecessary storage costs, minimize stockouts, and make better use of warehouse resources — staying ahead of supply chain disruptions and responding more effectively to changes in customer needs.*

# Why This Matters

## Inventory Cost Reduction

Through better stock alignment across all four warehouses

## Improved Service Level

Reducing stockouts in high-demand items

## Operational Efficiency

Better replenishment and warehouse planning

## Reduced Excess Inventory

For low-demand or intermittent products

## Stable Supply Chain Coordination

Enhanced operational efficiency across teams

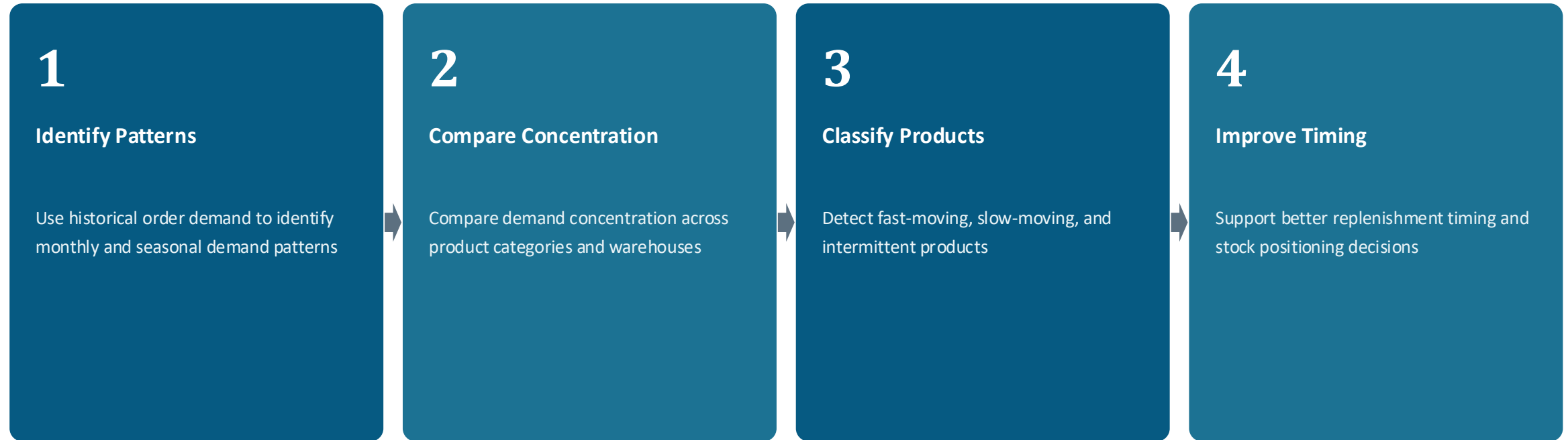
## Customer Satisfaction

Improved through better product availability



# The Data-Driven Opportunity

*This project bridges descriptive analytics to forecasting and inventory planning:*



*Creates a bridge from descriptive analytics to forecasting and inventory planning — exactly the path this project follows, chapter by chapter.*

# Some Insights From Data

*Early signals from the dataset already point toward concrete planning actions:*

## High-demand categories

May justify tighter review cycles and higher service targets

## Slow-moving products

May require lower stock levels or alternative replenishment strategies

## Warehouse demand imbalance

May indicate opportunity for stock reallocation

## Products with unstable demand

May need higher safety stock than products with stable demand

## Monthly demand patterns

Can support pre-season replenishment instead of last-minute reaction

## Product-code-level analysis

Helps identify which items drive most of the operational workload

# Stakeholders



## Inventory Managers

Need better stock level decisions and safety stock planning



## Operations & Production

Need visibility into expected demand to plan production and replenishment



## Sales Team

Need product availability to avoid lost opportunities and improve customer satisfaction



## Management & Decision-Makers

Need clear KPIs, risk visibility, and business impact estimates

# 02.2

## Theoretical Background

*Demand Management · Forecasting vs. Planning · Intermittent Demand · Time Series · Inventory Concepts*

# What Is Demand Management?

*Demand Management is the process of analyzing, forecasting, and influencing demand — helping align supply with customer needs.*

## Goal

Balance availability and cost

## Outcome

Ensures the right product is available at the right time

## Core Tool

Demand forecasting — estimating future demand from historical data

## Why

Reduces uncertainty and improves overall supply chain efficiency

*Estimating future demand from historical data and analytical methods directly reduces the uncertainty that drives stockouts and excess inventory.*

# Importance of Demand Management

## **Reduces stockouts and lost sales**

Customers find what they need, when they need it

## **Minimizes excess inventory and holding costs**

Capital isn't tied up in unsold stock

## **Improves customer satisfaction**

Reliable availability builds trust and repeat business

## **Supports better production and inventory planning**

Manufacturing schedules stay stable and predictable

## **Without It**

- Stockouts and lost revenue
- Excess capital tied up in inventory
- Unstable manufacturing schedules
- Disruptions that are hard to recover from in a long-lead-time global supply chain

# Forecasting vs. Demand Planning

*These two terms are often used interchangeably — but they describe different stages of the same process.*

## Forecasting

Predict future demand based on historical data.

**= Data-Driven**

*In this project: ARIMA, Random Forest, SARIMA, XGBoost, and Prophet all sit in this box — they produce a number.*

## Demand Planning

Uses forecasts + business knowledge to make decisions (sensing, planning, and coordination).

**= Decision-Making Process**

*In this project: turning a forecast number into a stocking decision — how much, when, where — is demand planning.*

# Intermittent Demand

## What It Is

- Demand occurs irregularly over time
- Many periods have zero demand
- Common in products that are not ordered regularly
- Difficult to forecast using standard methods

*Intermittent demand products should not be managed with the same logic as fast-moving products.*

## In Our Dataset

Product\_1341 is a clear example — its demand collapses structurally in 2016, which is exactly the kind of pattern intermittent-demand theory warns standard models will miss.

---

*This is why we apply ABC analysis and per-product model selection rather than one forecasting method for everything.*



# Characteristics of Intermittent Demand

## High Number of Zero-Demand Periods

Most time intervals show no orders at all — the series is mostly flat at zero

## Sudden Spikes in Demand

Large, infrequent orders breaking an otherwise quiet pattern

## Unpredictable Timing

No reliable cycle to anchor a forecast on — spikes don't follow a calendar

## High Variability

Order sizes swing widely whenever demand does occur

# Time Series Data

## Definition

- Data collected over time intervals
- Example: daily, weekly, or monthly demand
- Used to identify patterns and trends
- Important for demand analysis and forecasting

## In Our Project

Order-level demand records are aggregated into a monthly time series per warehouse and per top-4 product. Each series spans 2012–2016: 48 months training, 12 months holdout.

*Every model in this project — ARIMA, Random Forest, and the team's other models — is built on exactly this kind of structured time series.*

# Trend and Seasonality

## Trend

Long-term increase or decrease in demand over time.

**In our data:** Whse\_J trends upward across 2012–2016; Whse\_S trends downward.

## Seasonality

Repeating patterns over time (e.g. weekly, yearly cycles).

**In our data:** All four warehouses show a repeating 12-month seasonal cycle.

*Together, trend and seasonality help explain why demand looks the way it does — before any model is even fit.*

# Demand Variability

- Demand does not stay constant
- Can change due to customer behavior or external factors (weather, economy)
- High variability increases planning difficulty
- Directly affects inventory risk and service performance

## Why It Matters Here

Whse\_S shows the clearest example: a sustained decline through 2016 that neither ARIMA nor Random Forest fully captures — their MAPE (45.2% and 36.6%) is the highest of any warehouse precisely because variability is highest there.

---

*Variability isn't noise to ignore — it's a signal of which series need more safety stock and more careful model selection.*

# Inventory Concepts — Safety Stock



## Safety Stock

- Extra inventory kept to avoid stockouts
- Used when demand is uncertain
- Helps maintain service level

**Too much** → high cost    **Too little** → risk of shortage

### Connecting to Our Findings

Whse\_S's residuals from Random Forest are almost entirely negative through 2016 — a systematic over-forecasting bias. That's a textbook signal that safety stock there should be recalibrated, not just increased blindly.

---

*Products with unstable demand (e.g. Product\_1341) need higher safety stock than products with stable demand.*

# Inventory Concepts — Lead Time



## Lead Time

- Time between ordering and receiving products
- Includes production and delivery time
- Important for planning replenishment
- Longer lead time = higher uncertainty

### Why This Project Exists

Our dataset's products ship via ocean freight with a lead time exceeding one month. That gap between ordering and receiving is precisely why forecasting matters — by the time demand is visible, it's already too late to react.

*This is the theoretical foundation for everything that follows in Dataset and ML Algorithms.*

# From Theory to Practice

*Every concept on the last 10 slides maps directly onto a decision made later in this project:*

**Time Series Data, Trend & Seasonality**



Exploratory data analysis & decomposition

**Demand Variability**



ADF stationarity testing per warehouse

**Intermittent Demand**



ABC analysis & top-4 product selection

**Forecasting vs. Demand Planning**



Model outputs (ARIMA, RF...) vs. inventory decisions

**Safety Stock & Lead Time**



Business Impact framing & recommendations

*Next up: the Dataset chapter — where these theoretical concepts meet 1,048,575 real transaction rows.*

## 03

# Dataset

*Source · Description & Data Understanding · Data Preparation · Descriptive Data Analytics*



# Dataset Source

## Forecasts for Product Demand

*Published on Kaggle by Felix Zhao*

|           |                            |
|-----------|----------------------------|
| Platform  | Kaggle                     |
| License   | GPL v2                     |
| Format    | Single CSV file            |
| Time span | 2011 – 2017                |
| Size      | 1,048,575 rows × 5 columns |

[kaggle.com/datasets/felixzhao/productdemandforecasting](https://kaggle.com/datasets/felixzhao/productdemandforecasting)

```
import pandas as pd

df = pd.read_csv(
    'Historical_Product_Demand.csv'
)
print(df.shape)

>>> (1048575, 5)
```

The data originates from a global manufacturing company supplying thousands of products across multiple categories — a real-world source, which makes it more representative of actual business challenges than synthetic data.

# Dataset Overview

**1,048,575**

transaction rows

**5**

columns

**2,160**

unique products

**33**

product categories

**4**

warehouses

**2011–2017**

date range

*Every row represents one customer order for one product on one date at one warehouse — the rawest possible grain of demand data, which is exactly what makes it useful for forecasting.*

# Feature Description

| Column           | Type                 | Scale    | Description                                              |
|------------------|----------------------|----------|----------------------------------------------------------|
| Product_Code     | object (categorical) | Nominal  | Unique identifier per product, e.g. Product_0993         |
| Warehouse        | object (categorical) | Nominal  | One of 4 warehouses: Whse_A, Whse_C, Whse_J, Whse_S      |
| Product_Category | object (categorical) | Nominal  | One of 33 categories, e.g. Category_028                  |
| Date             | object → datetime    | Interval | Order date; 11,239 missing values                        |
| Order_Demand     | object → numeric     | Ratio    | Quantity ordered; stored as text incl. negative notation |

Scale type matters for analysis choice: nominal columns drive grouping/segmentation, while Order\_Demand (ratio scale) is the target variable for every forecasting model in this project.

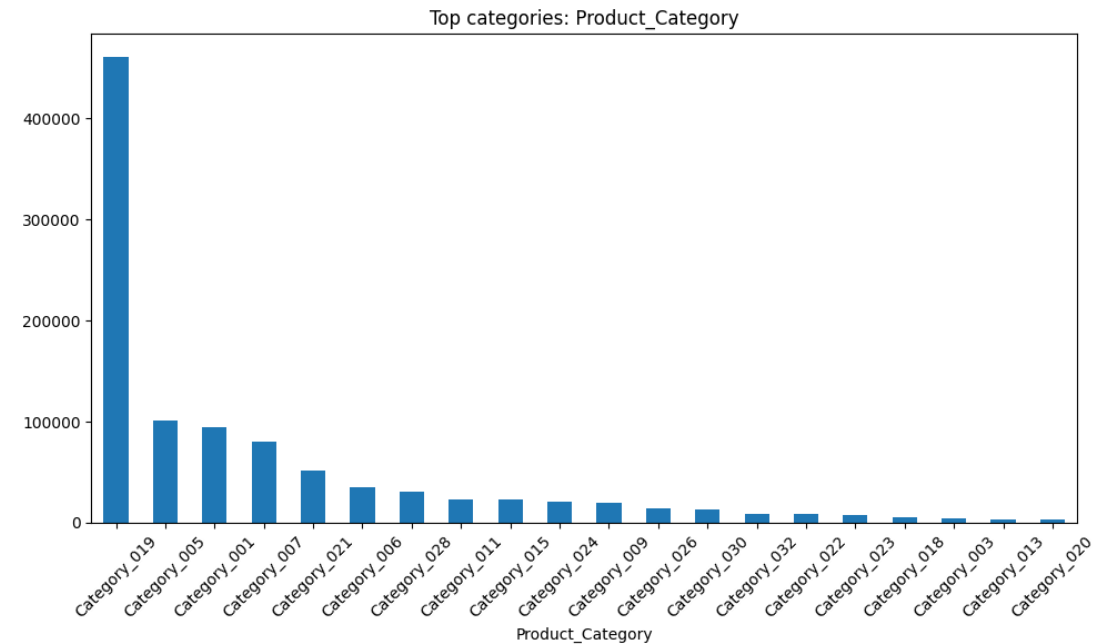
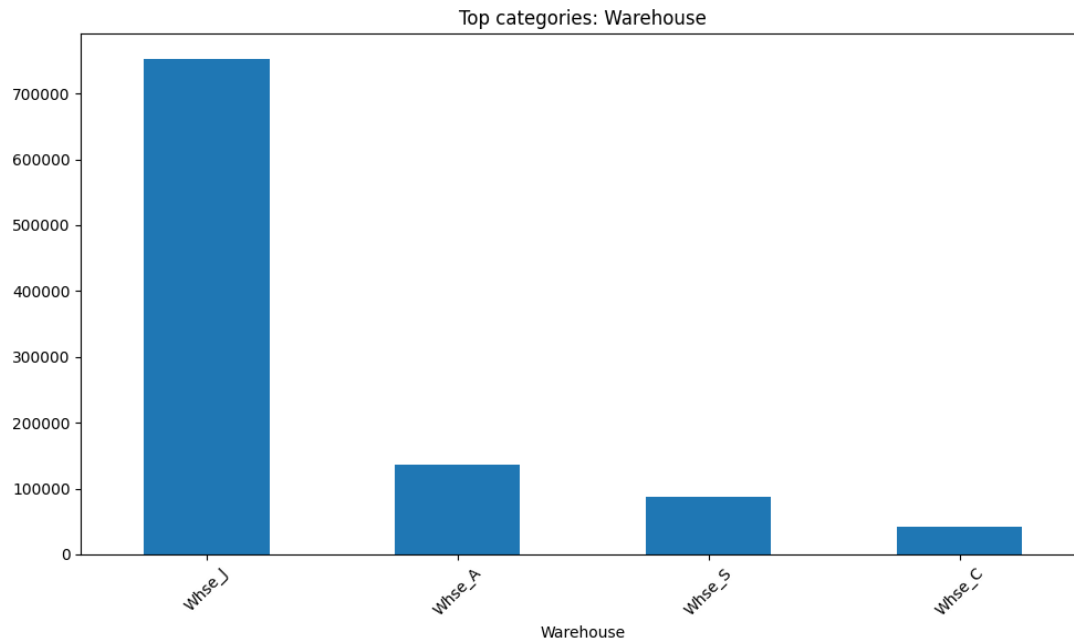
# Sample Data

```
print(df.head())
print(df.dtypes)
print(df.isnull().sum())
```

| Product_Code | Warehouse | Product_Category | Date      | Order_Demand |
|--------------|-----------|------------------|-----------|--------------|
| Product_0993 | Whse_J    | Category_028     | 2012/7/27 | 100          |
| Product_0979 | Whse_J    | Category_028     | 2012/1/19 | 500          |
| Product_1346 | Whse_J    | Category_023     | 2012/8/1  | 300          |
| Product_0417 | Whse_J    | Category_021     | 2012/2/27 | (1)          |

*Note the last row: Order\_Demand “(1)” — parenthesis notation, a recurring pattern we deal with in Data Preparation.*

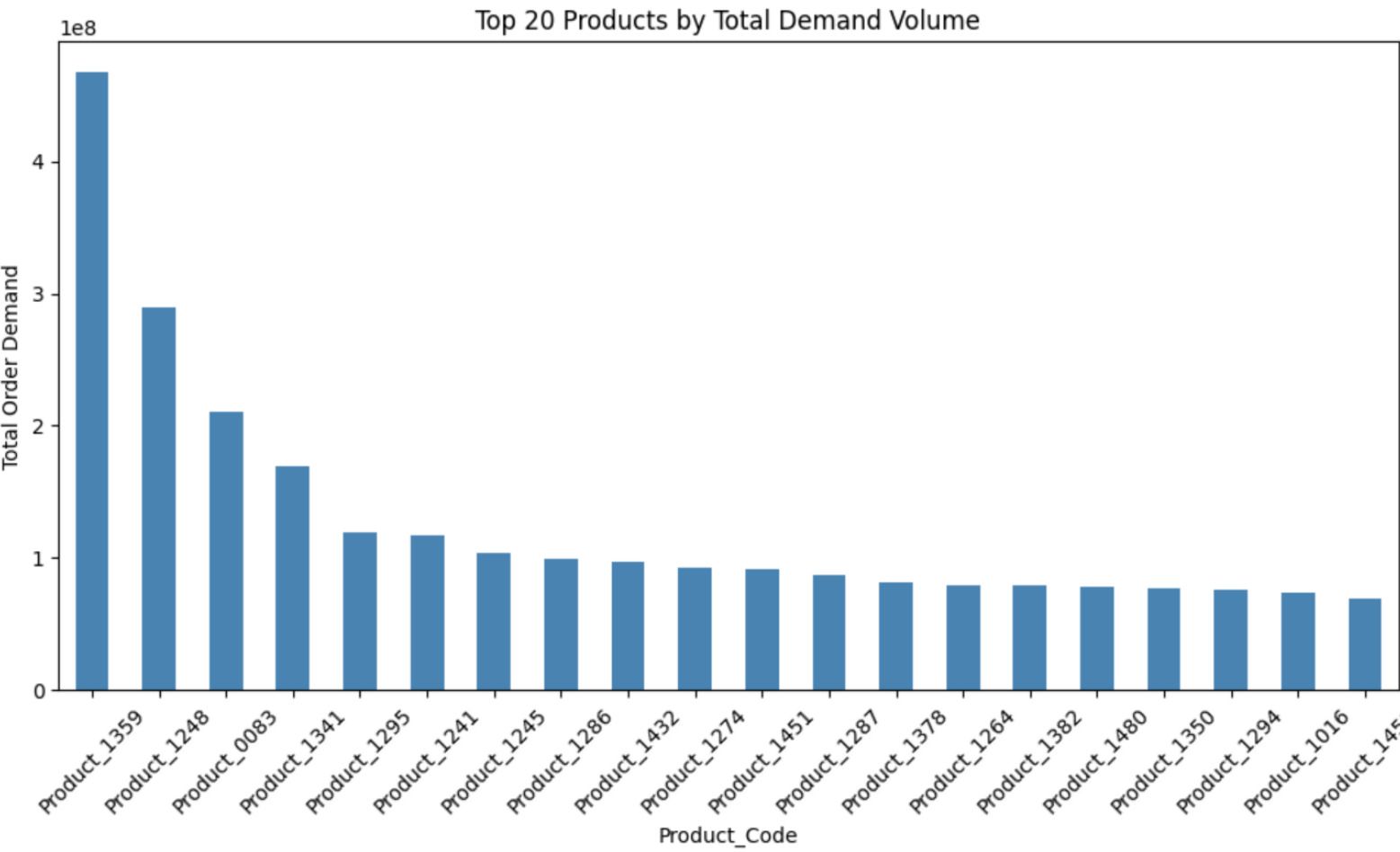
# Categorical Column Distributions



```
cat_cols = df_new.select_dtypes(include=['object', 'category']).columns
for col in cat_cols:
    plt.figure(figsize=(10, 6))
    df_new[col].value_counts().head(20).plot(kind='bar')
    plt.title(f'Top categories: {col}')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

*Warehouse (left) and Product\_Category (right) are both heavily skewed — Whse\_J dominates by row count, and Category\_019 alone accounts for nearly half of all records.*

# The Most Ordered Products



## Order Volume

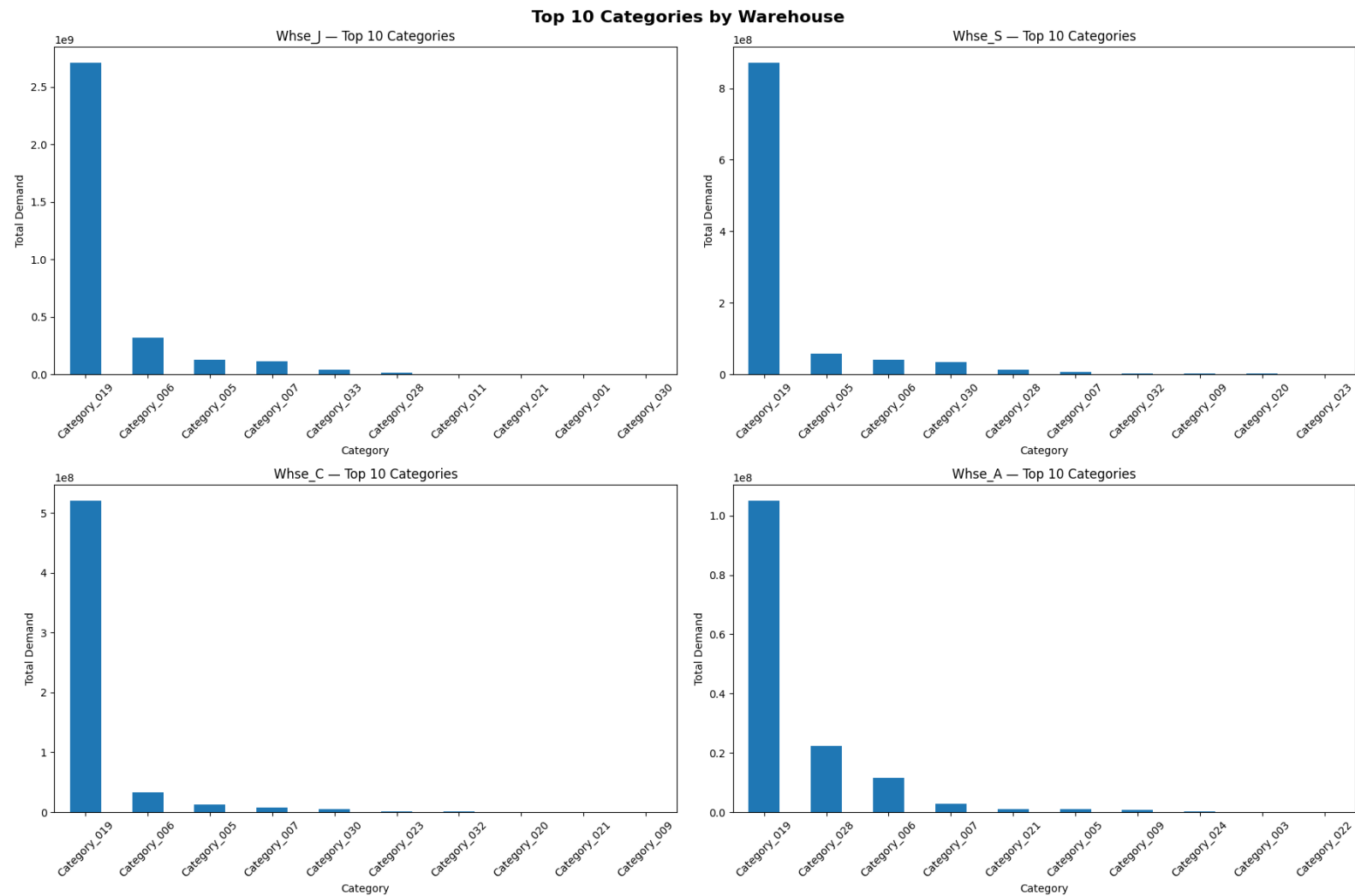
Product\_1359 is the most ordered product and the most frequent order as well.

Product 1359, 1248, 0083, 1341 are the highest volumes among Top 20.

Volume tells us how much.

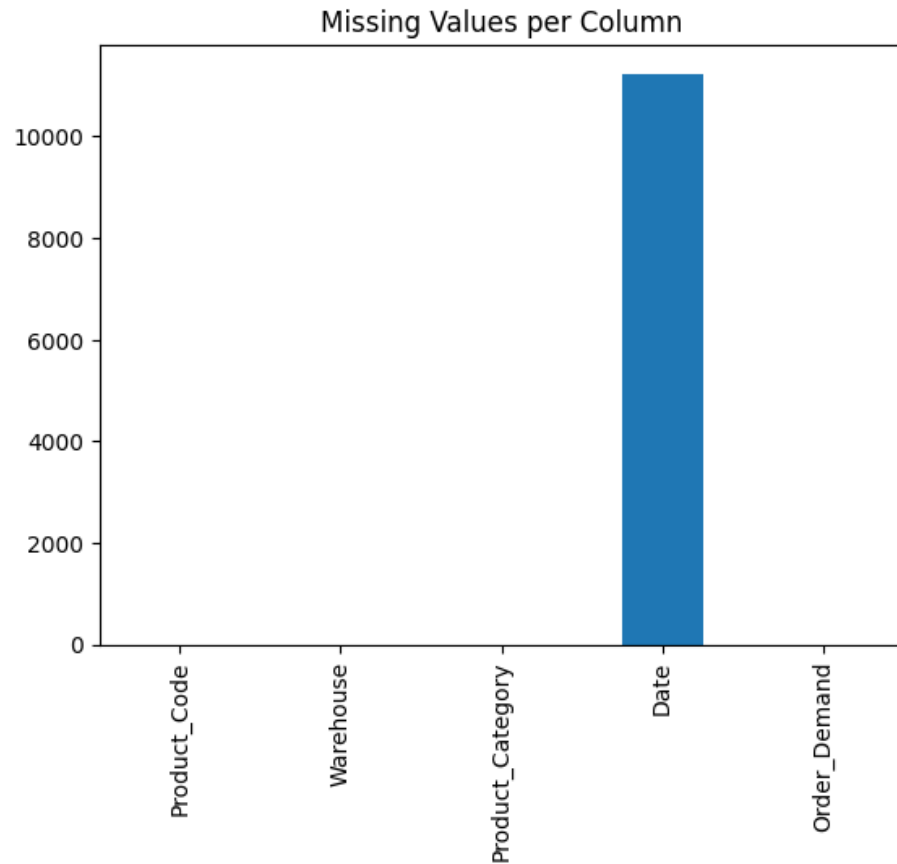
Frequency tells us how often we need to plan for a product. Both matter for replenishment.

# Category Concentration Holds Across Every Warehouse



*Category\_019 is the top category in all four warehouses — this is a structural pattern in the business, not a fluke of one location.*

# Data Quality Assessment



```
import matplotlib.pyplot as plt
missing = df.isnull().sum()

missing.plot(kind='bar')
plt.title("Missing Values per Column")
plt.show()
```

## Missing dates

11,239 rows (1.1%) have no Date value

## Stringified numbers

Order\_Demand stored as text, not numeric

## Negative notation

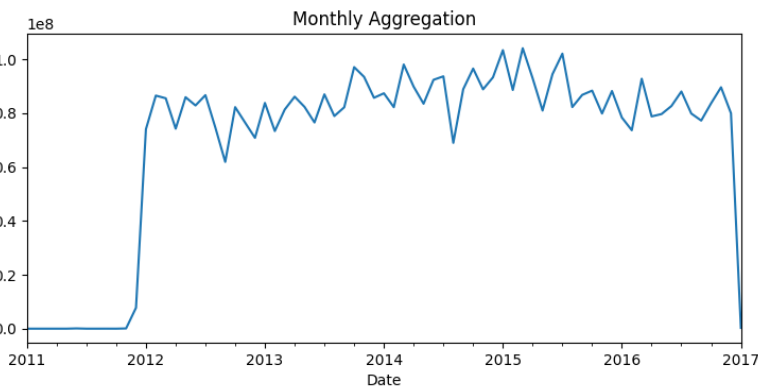
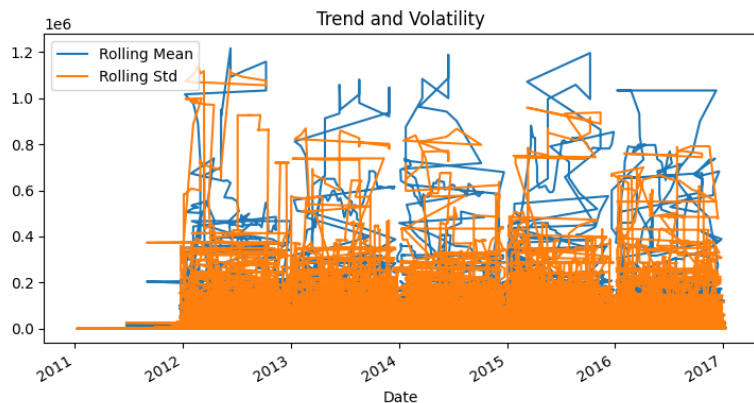
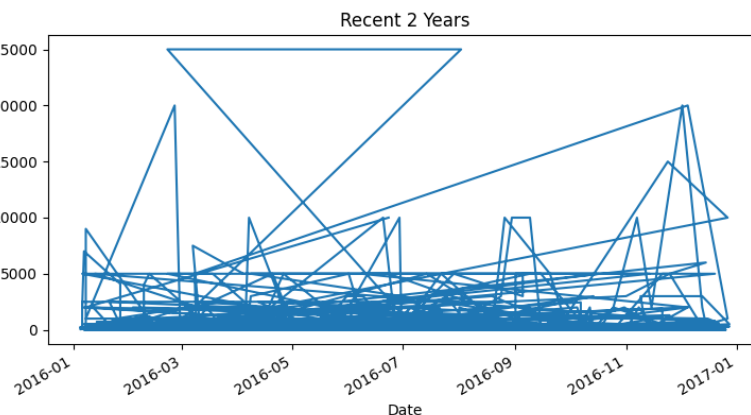
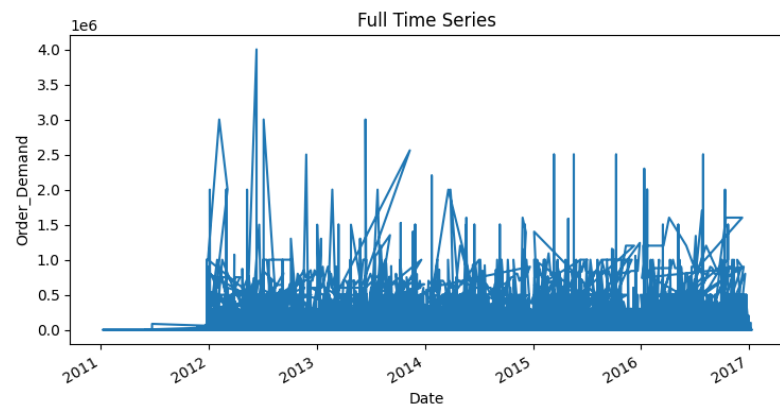
Parentheses denote cancellations/returns, e.g. (100)

## Extreme duplicates

Some rows repeat  $\geq 5$  times identically



# Why Raw Transaction Data Isn't Forecast-Ready

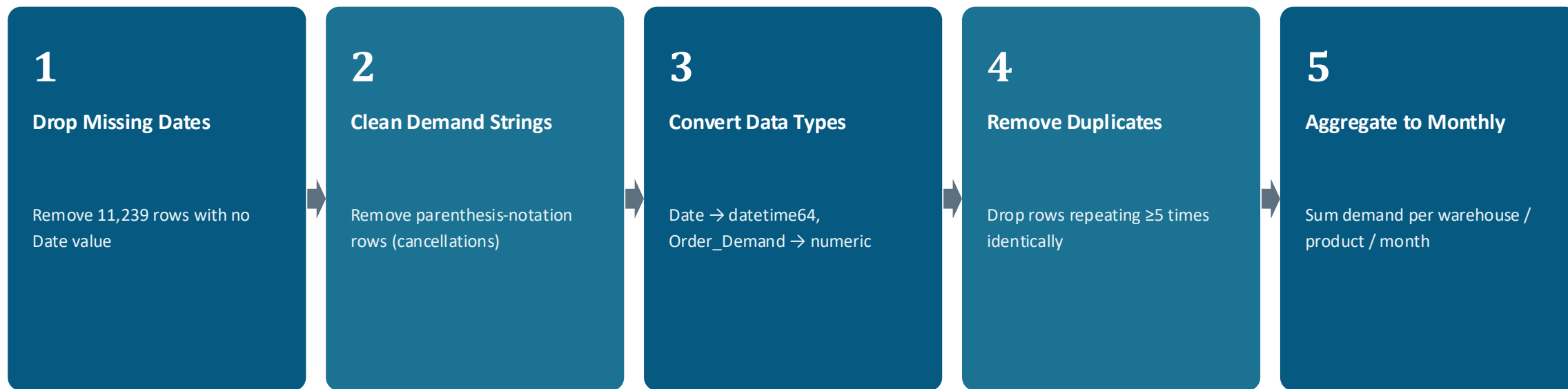


**At transaction grain, the signal is noise**

Individual orders are too sparse and erratic to forecast directly. Patterns only emerge once data is aggregated into a consistent time grain.

*Monthly aggregation (bottom-right panel) is what turns chaos into a usable signal — this motivates every step in Data Preparation next.*

# Cleaning Pipeline Overview



**1,048,575 rows in → demand\_cleaned.csv out**

*Each step is applied in sequence, with row counts logged before and after, so every removal is auditable and explainable to the professor.*

# Step 1 — Handling Missing Dates

```
print(df['Date'].isnull().sum())
>>> 11239

df_cleaned = df.dropna(subset=['Date'])

print(df_cleaned.shape)
>>> (1037336, 5)
```

## Why drop instead of impute?

A missing order date cannot be reliably inferred — imputing it (e.g. with a mean or forward-fill) would fabricate a time-series position for real demand, distorting monthly aggregation. At just 1.1% of rows, dropping is the safer choice.

### Before → After

Rows

**1,048,575 → 1,037,336**

Missing dates

**11,239 → 0**

% removed

**— → 1.07%**

## Step 2 — Handling Stringified Demand Values

```
# Identify parenthesis-notation rows
neg_mask = df_cleaned['Order_Demand'].str.contains(
    '\(', regex=True
)
print(neg_mask.sum())
>>> 5899
```

```
# Remove them, then convert to numeric
df_cleaned = df_cleaned[~neg_mask]
df_cleaned['Order_Demand'] = pd.to_numeric(
    df_cleaned['Order_Demand']
)
```

**Rationale:** parenthesis notation is standard accounting convention for negative values — these rows likely represent cancellations or returns, which would distort true demand if kept. (Kieso et al., 2020)

### Before → After

Rows

**1,037,336 → 1,031,437**

"(...)" rows

**5,899 → 0**

% removed

**— → 0.57%**

## Step 3 — Removing Duplicates & Finalizing Types

```
# Flag extreme duplicate rows (>=5 identical occurrences)
dup_counts = df_cleaned.groupby(
    list(df_cleaned.columns)
).transform('size')

df_cleaned = df_cleaned[dup_counts < 5]

# Confirm final dtypes
print(df_cleaned.dtypes)
print(df_cleaned.isnull().sum())

>>> 0 nulls, correct dtypes throughout
```

### Why 5+ occurrences specifically?

*A small number of repeats can be genuine (same product reordered identically); 5+ identical rows is far more likely to be a logging artifact than real demand.*

### Final Dataset Shape

*demand\_cleaned.csv*

|                  |                |
|------------------|----------------|
| Product_Code     | object         |
| Warehouse        | object         |
| Product_Category | object         |
| Date             | datetime64[ns] |
| Order_Demand     | int64          |

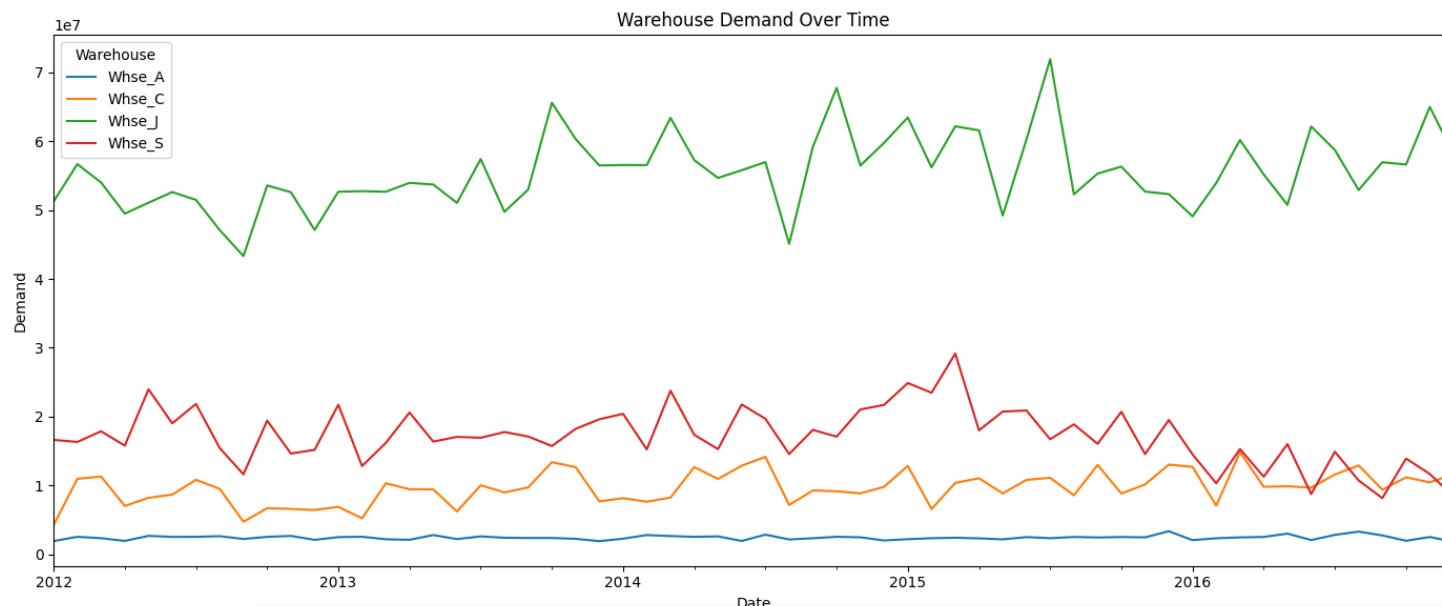
# Before & After — Cleaning Summary

| Stage                             | Rows Remaining | Rows Removed | Reason                    |
|-----------------------------------|----------------|--------------|---------------------------|
| Raw data                          | 1,048,575      | —            | —                         |
| After dropping missing dates      | 1,037,336      | 11,239       | No reliable time position |
| After removing "(...)" rows       | 1,031,437      | 5,899        | Cancellations / returns   |
| After removing extreme duplicates | ~1,030,000*    | ~1,400*      | Likely logging artifacts  |

\*Approximate — exact duplicate count depends on final groupby key definition; logged precisely in the notebook output.

**Net result:** ~98.2% of the original data survives cleaning, with full traceability of every row removed and why — ready for monthly aggregation and modeling.

# Demand by Warehouse Over Time



- Whse\_J handles the large majority of demand and trends upward across 2012–2016
- Whse\_A is a distant second, relatively flat
- Whse\_S shows a visible downward drift through 2015–2016
- Whse\_C stays smallest and most stable

```
import matplotlib.pyplot as plt

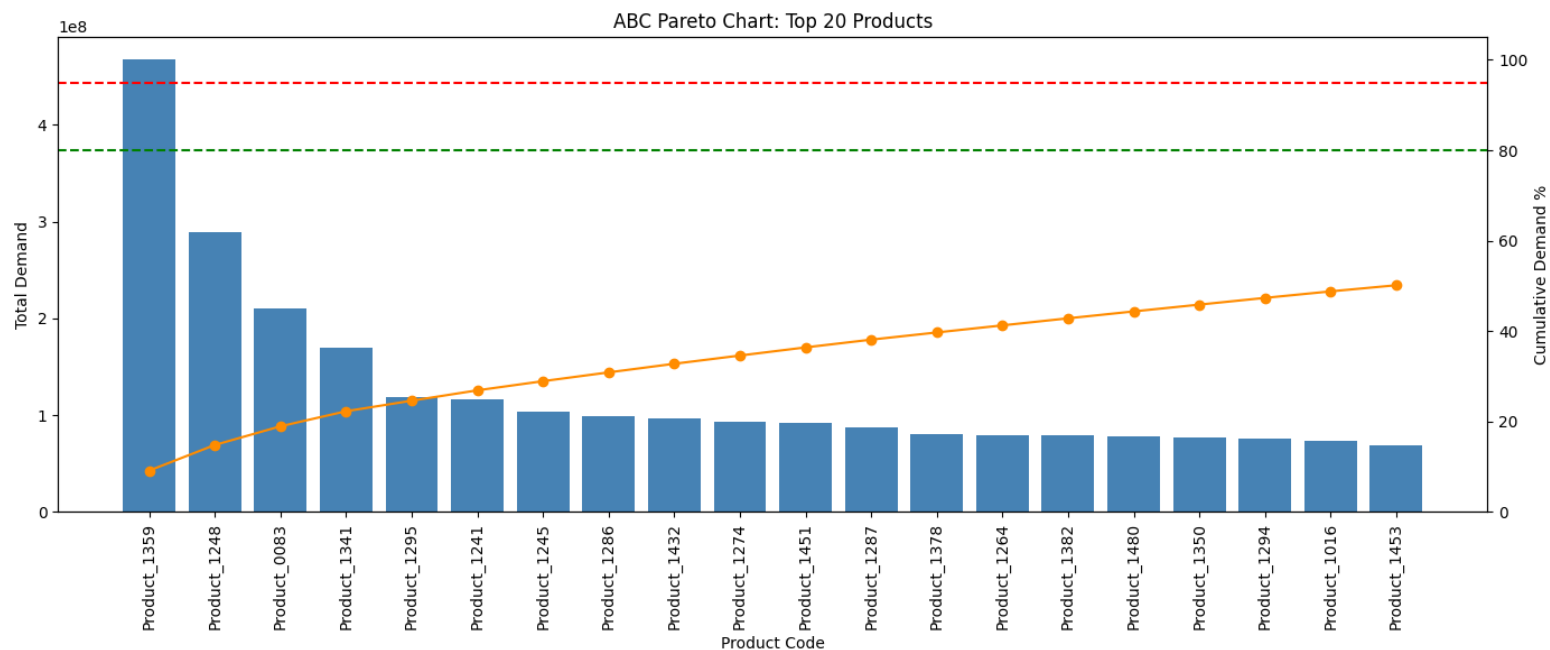
df_plot = df_new[(df_new['Date'].dt.year >= 2012) &
                 (df_new['Date'].dt.year <= 2016)]

wh_month = df_plot.groupby([df_plot['Date'].dt.to_period('M'),
                           'Warehouse'])['Order_Demand'].sum().unstack()

plt.tight_layout()
plt.show()
```

*This concentration and divergence is the first clue that one model won't fit all four warehouses equally well.*

# Top Products by Demand Volume



## Reading the Pareto Chart

Bars show total demand per product (left axis); the orange line tracks cumulative share of total demand (right axis). Where the line crosses the green 80% mark identifies the “vital few” products.

*Product\_1359 alone accounts for ~9.2% of total demand — the single highest-volume product in the dataset, and our top pick for product-level modeling.*

```
abc = (df_new.groupby('Product_Code')['Order_Demand'].sum().sort_values(ascending=False)
.reset_index())plt.show()
abc['cum_demand_pct'] = abc['Order_Demand'].cumsum() / abc['Order_Demand'].sum() * 100
abc['ABC_Class'] = abc['cum_demand_pct'].apply(classify_abc)

fig, ax1 = plt.subplots(figsize=(14, 6))
top_n = 20
plot_df = abc.head(top_n)
```

```
def classify_abc(x):
    if x <= 80:
        return 'A'
    elif x <= 95:
        return 'B'
    else:
        return 'C'
```



# ABC Classification

*Products are grouped into three tiers based on cumulative demand contribution — a standard inventory-management technique:*

## A

**Top ~20% of products**

**~80% of total demand**

---

Tight control, high service targets, frequent review

## B

**Next ~30% of products**

**~15% of total demand**

---

Moderate control, standard review cycles

## C

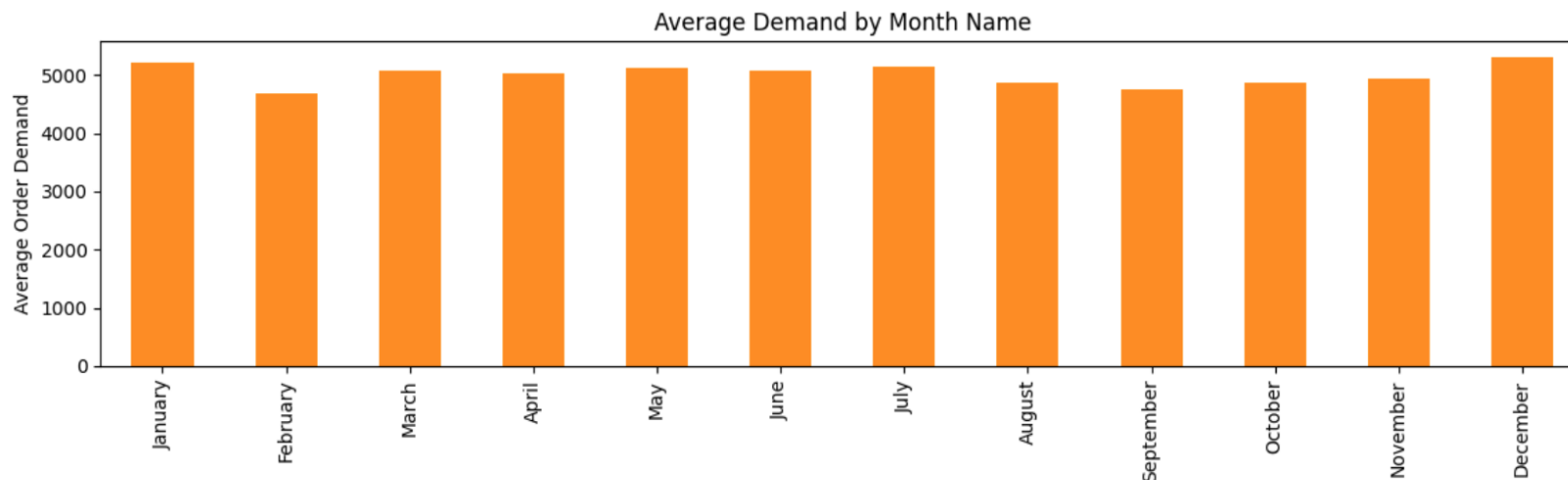
**Remaining ~50% of products**

**~5% of total demand**

---

Light control, infrequent review, simple rules

# Seasonality — Average Demand by Month



```
df_new['Month_Name'] = df_new['Date'].dt.month_name()
month_avg = df_new.groupby('Month_Name')['Order_Demand'].mean().reindex([
    'January', 'February', 'March', 'April', 'May', 'June',
    'July', 'August', 'September', 'October', 'November', 'December'])
```

```
plt.figure(figsize=(12, 4))
month_avg.plot(kind='bar', color='darkorange')
plt.title('Average Demand by Month Name')
plt.xlabel('Month')
plt.ylabel('Average Order Demand')
plt.tight_layout()
plt.show
```

## Finding

Demand is not evenly distributed across months — March, July, and October show peaks, while January, August, and September dip lower.

## Business Interpretation

- Peak months need extra safety stock & proactive procurement
- Low months are opportunities to reduce holding cost
- The 12-month regularity is one of the most actionable EDA insights

# What Is Time Series Decomposition?

Time series decomposition breaks down each warehouse's monthly demand into three components to better understand the underlying demand behavior:

## Trend

The long-term direction of demand — increasing, decreasing, or stable over time.

## Seasonality

Repeating patterns that occur at regular intervals — every 12 months in our monthly data.

## Residual

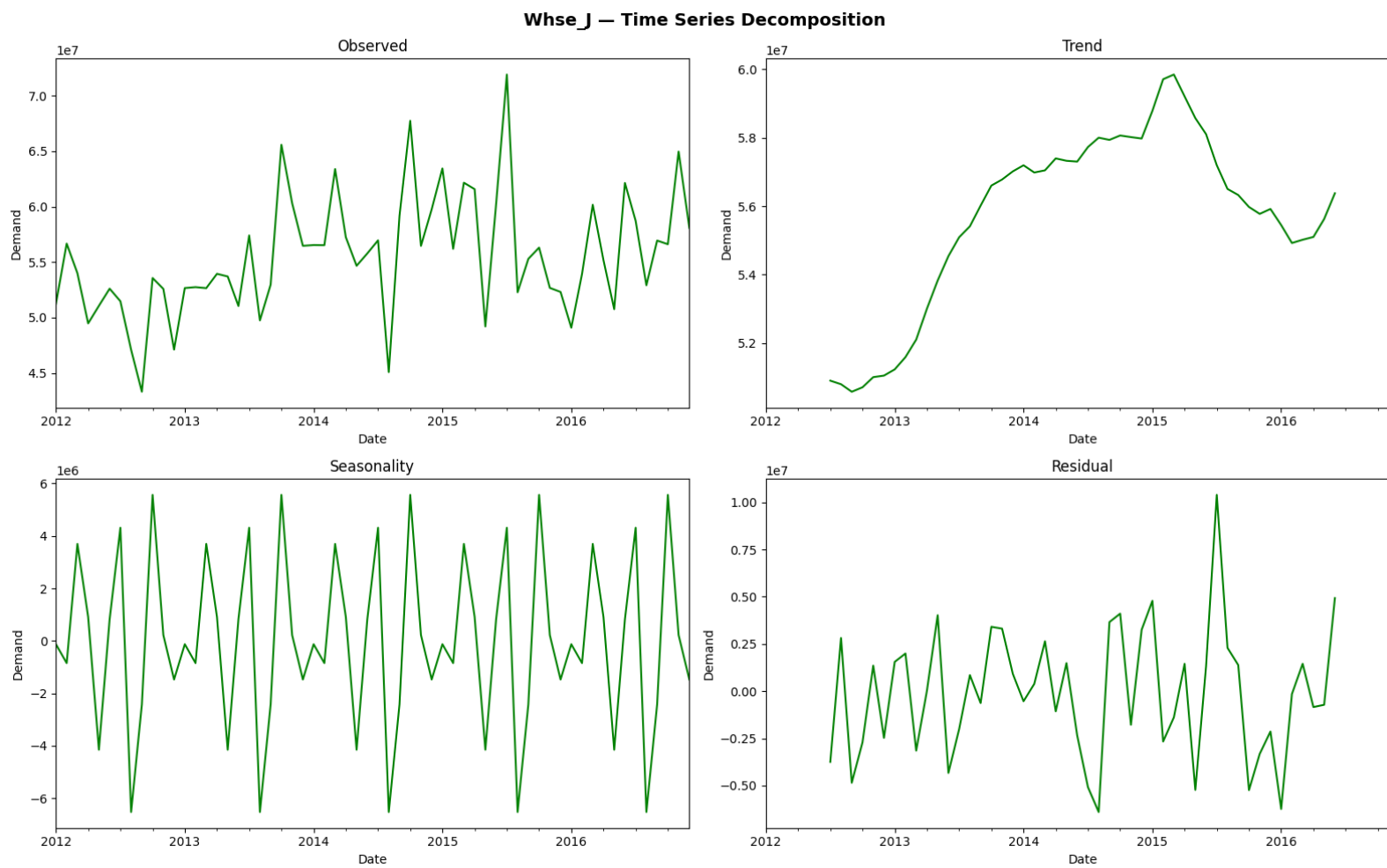
Random noise left over after the trend and seasonal components are removed.

### Additive model: $y_t = S_t + T_t + R_t$

Used here (vs. multiplicative) because seasonal swings stay roughly constant in size relative to the demand level, rather than growing with it (Liu, 2022, pp. 294–295). Computed with statsmodels' `seasonal_decompose(model='additive', period=12)`.

Reference: Liu, K. Y. (2022). *Supply Chain Analytics: Concepts, Techniques and Applications* (pp. 294–296). Palgrave Macmillan.

# Time Series Decomposition — Whse\_J (The best ARIMA fit)



- Trend: clear rise through 2014–2015, then a pullback
- Seasonality: sharp, repeating 12-month cycle
- Residual: mostly small, with one notable spike (mid-2015)

*This will be the best performs (5.5% MAPE) in ML Algorithms section — strong trend and seasonality are exactly what classical time-series models are built for.*

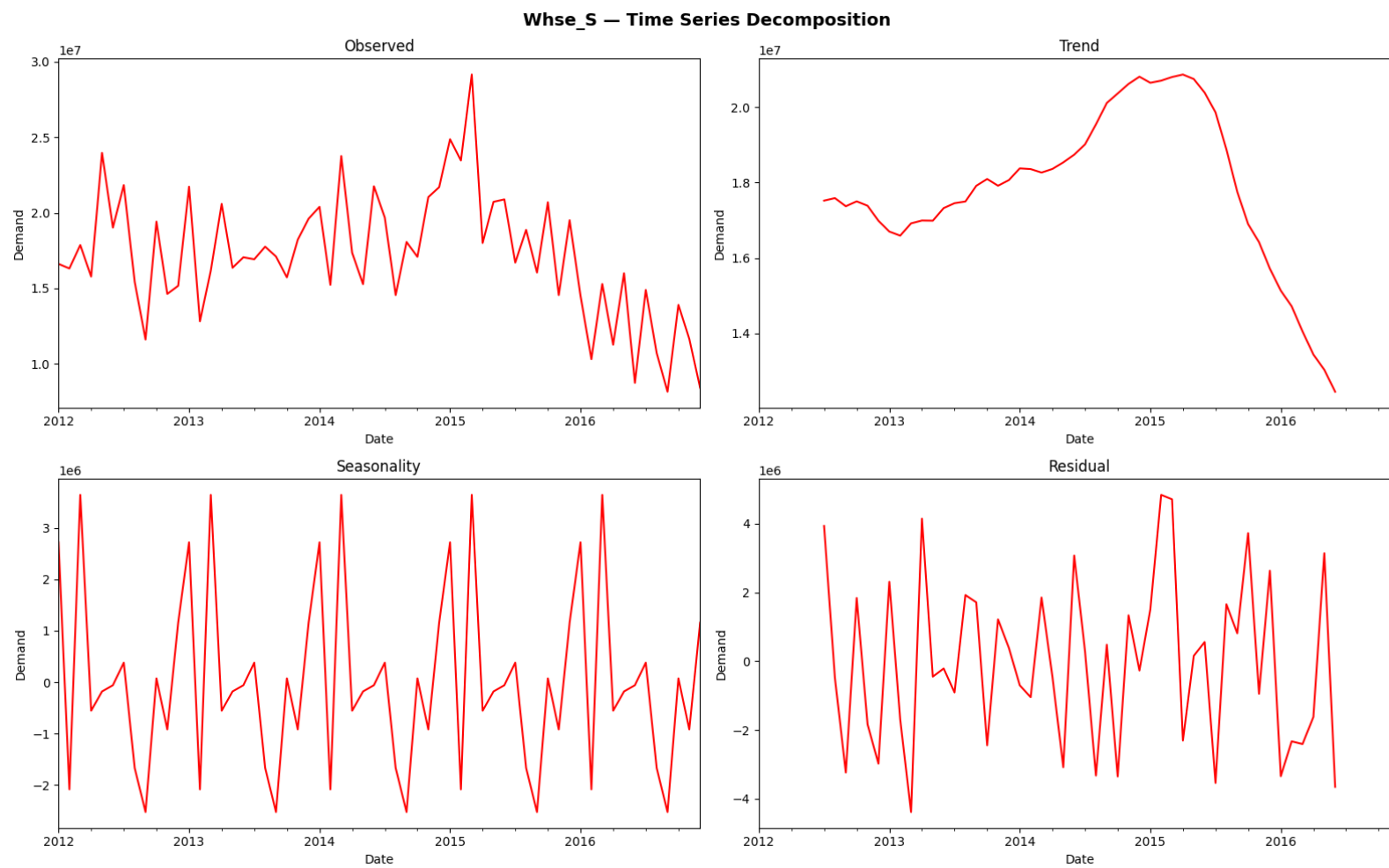
```
df_wh = df_new[(df_new['Warehouse'] == 'Whse_J') &
               (df_new['Date'].dt.year >= 2012) & (df_new['Date'].dt.year <= 2016)]

monthly = df_wh.groupby(df_wh['Date'].dt.to_period('M'))['Order_Demand'].sum()
monthly.index = monthly.index.to_timestamp()

decomposition = seasonal_decompose(monthly, model='additive', period=12)
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

plt.suptitle('Whse_J — Time Series Decomposition', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```

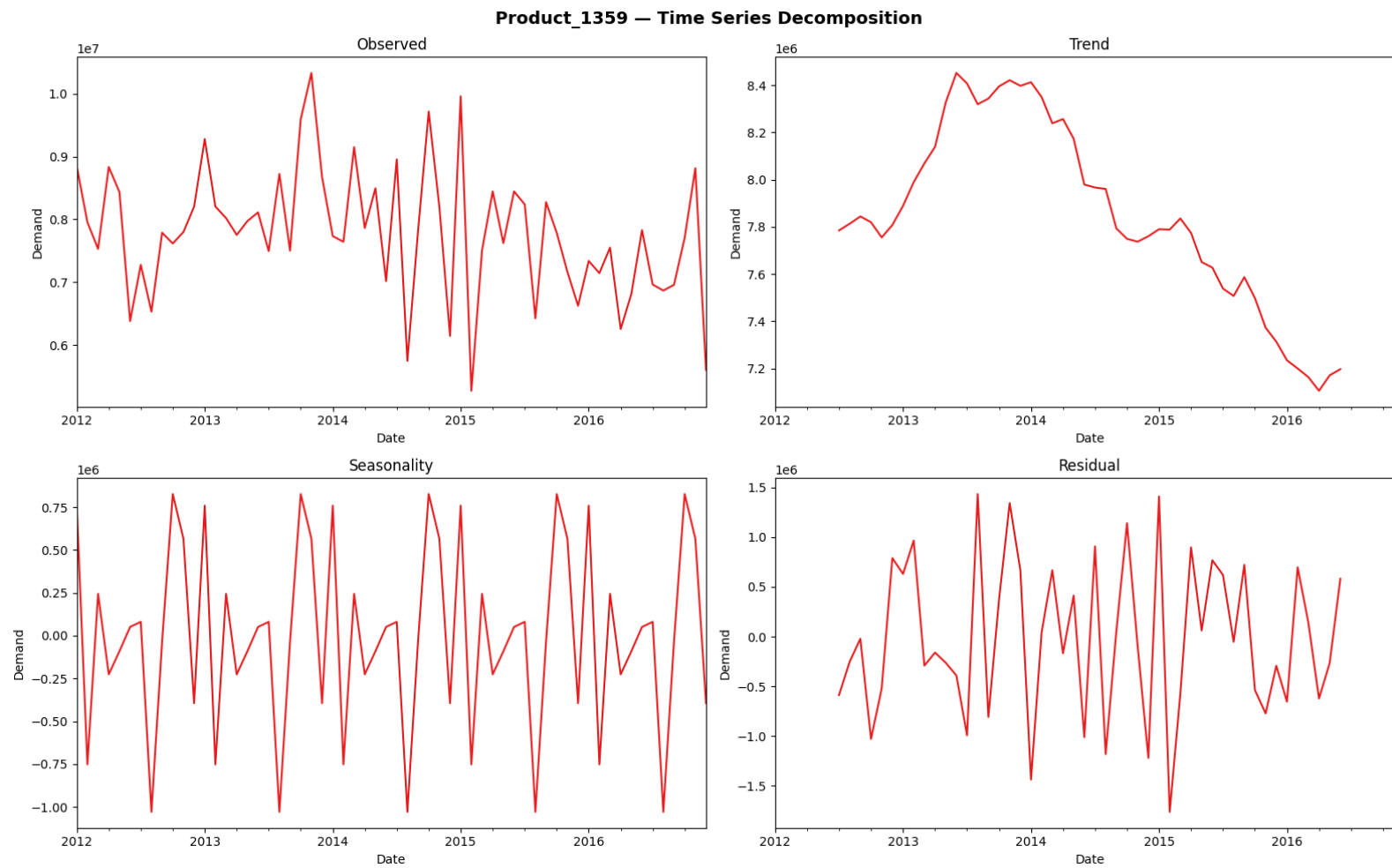
# Time Series Decomposition — Whse\_S (The worst ARIMA fit)



- Trend: rises until early 2015, then declines sharply through 2016
- Seasonality: similar 12-month shape to Whse\_J, smaller amplitude
- Residual: noisier, larger swings than Whse\_J

*This structural break is why Whse\_S is the hardest warehouse to forecast (45.2% ARIMA MAPE) — the decline is real, not noise.*

# Product-Level Decomposition — Product\_1359 (the top-selling product)



- Trend: rises into 2013–2014, then steadily declines
- Seasonality: very regular annual cycle, larger relative to demand level than the warehouse series
- Residual: substantial noise — product-level demand is inherently less smooth than aggregated warehouse demand

*Even our highest-volume product is noisier than any warehouse aggregate — a pattern that holds across all four top products.*

```
for prod in top4_codes:
    df_prod = df_new[(df_new['Product_Code'] == prod) &
                     (df_new['Date'].dt.year >= 2012) &
                     (df_new['Date'].dt.year <= 2016)]

    monthly =
df_prod.groupby(df_prod['Date'].dt.to_period('M'))['Order_Demand'].sum()
    monthly.index = monthly.index.to_timestamp()

    decomposition = seasonal_decompose(monthly,
model='additive', period=12)

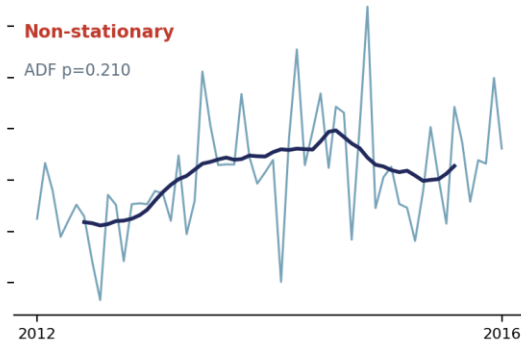
    fig, axes = plt.subplots(2, 2, figsize=(16, 10))
    plt.suptitle(f'{prod} — Time Series Decomposition',
fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

# Time Series Decomposition — All 8 Series

Decomposition (trend + seasonality + residual) was run for all 4 warehouses and all 4 top products, not just the 3 detailed on the previous pages (★).

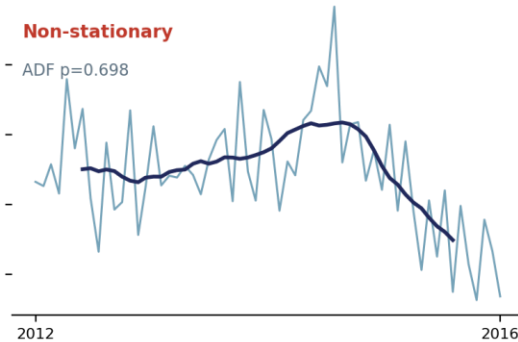
**Whse\_J ★**

- **Non-stationary**  
- ADF p=0.210



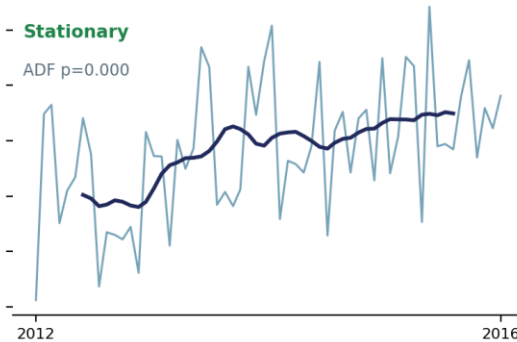
**Whse\_S ★**

- **Non-stationary**  
- ADF p=0.698



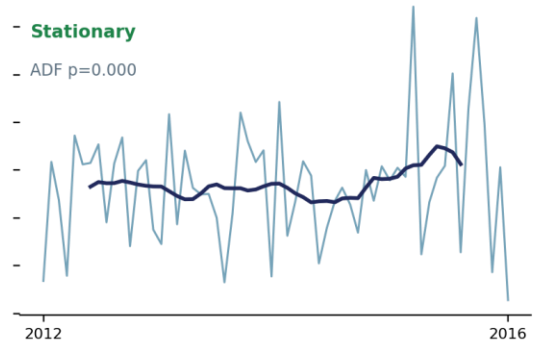
**Whse\_C**

- **Stationary**  
- ADF p=0.000



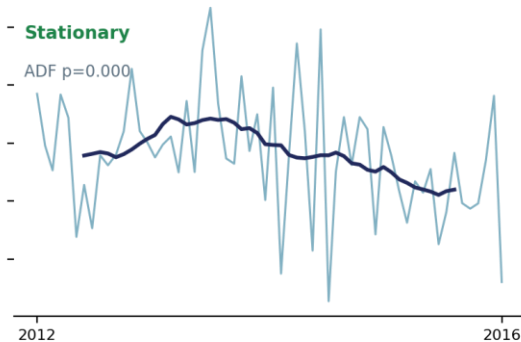
**Whse\_A**

- **Stationary**  
- ADF p=0.000



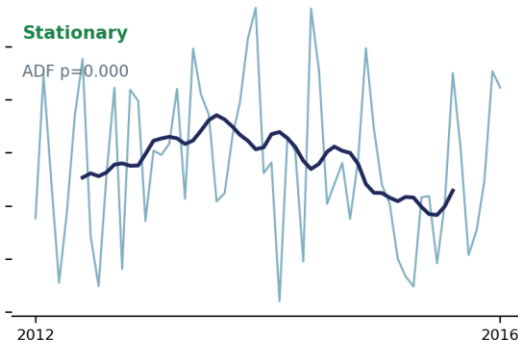
**Product\_1359 ★**

- **Stationary**  
- ADF p=0.000



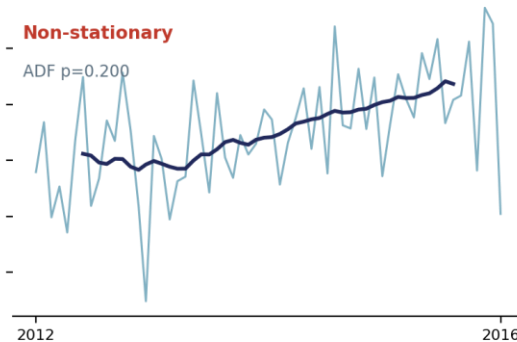
**Product\_1248**

- **Stationary**  
- ADF p=0.000



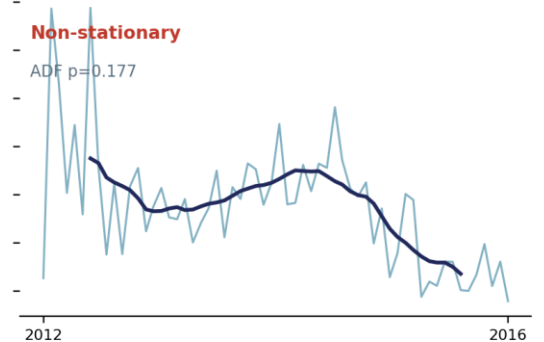
**Product\_0083**

- **Non-stationary**  
- ADF p=0.200



**Product\_1341**

- **Non-stationary**  
- ADF p=0.177



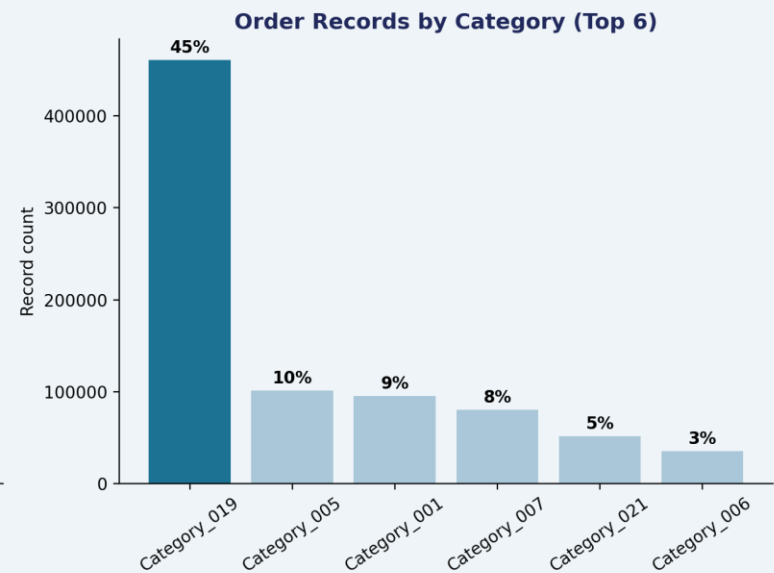
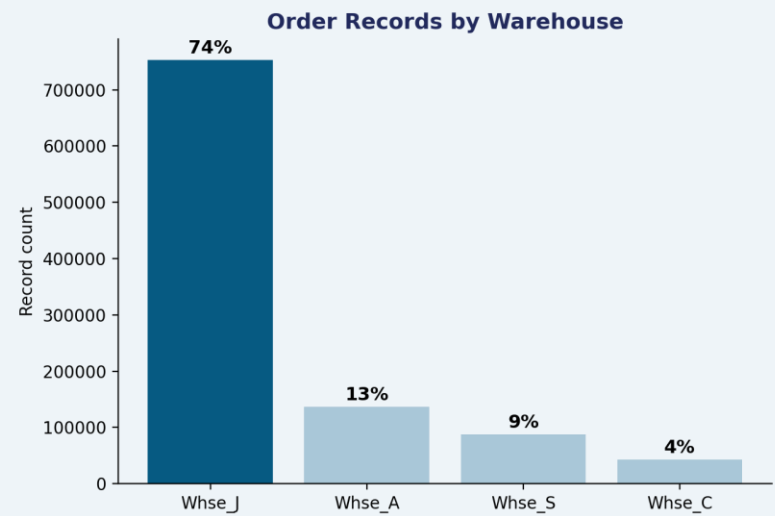
## Same stationary / non-stationary split drives every ARIMA order choice

Whse\_J, Whse\_S, Product\_0083 and Product\_1341 are non-stationary (d=1 in ARIMA); Whse\_C, Whse\_A, Product\_1359 and Product\_1248 are stationary (d=0, ARMA). This 50/50 split is exactly what motivated comparing classical and ML models across the whole portfolio, not just one series.

# Key EDA Insights — From Data to Modeling

## Highly concentrated supply chain

Whse\_J dominates demand with 74% of all order records; Category\_019 alone drives ~46% — and it's the top category in every one of the 4 warehouses, not just Whse\_J.



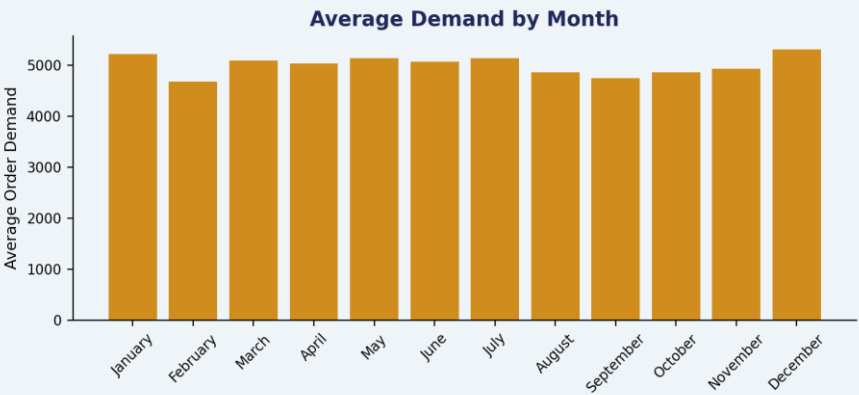
EDA: Exploratory Data Analysis



# Key EDA Insights — From Data to Modeling

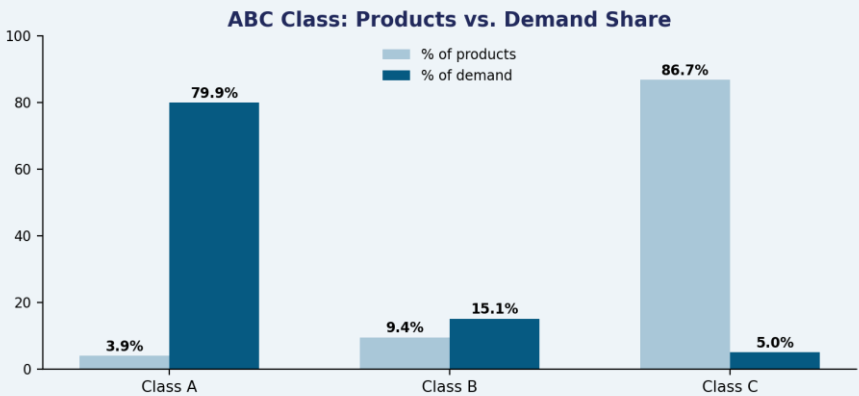
## Demand is not random

Average monthly demand follows a predictable 12-month cycle, with March, July and October peaking and January/August/September dipping lower.



## Demand is highly skewed (ABC)

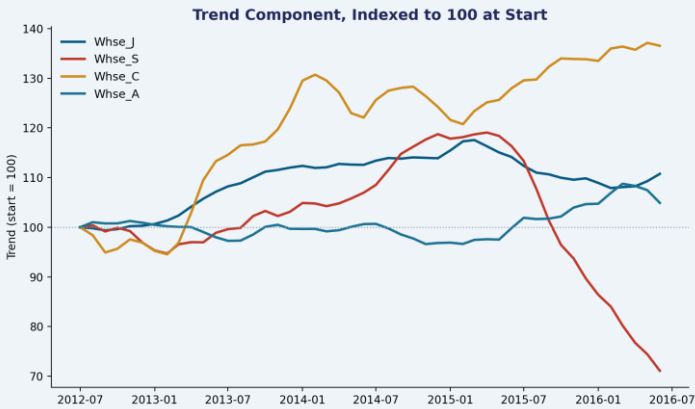
Class A is just 3.9% of products (85 of 2,160) but accounts for 79.9% of total demand — tight control on a small set of products drives most of the outcome.



# Key EDA Insights — From Data to Modeling

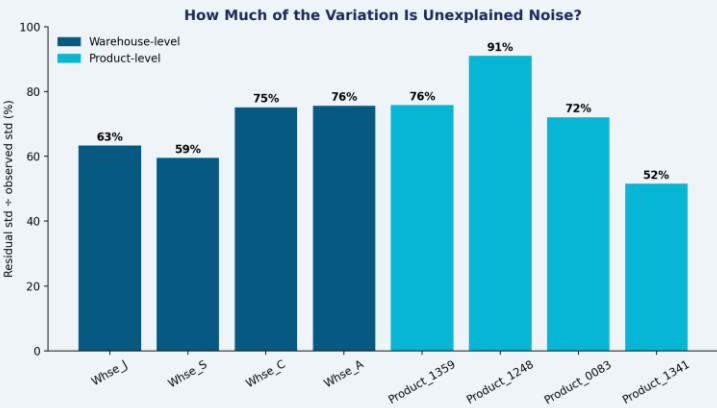
## Trend direction varies by warehouse

Indexing each warehouse's trend to 100 at the start shows genuinely different paths — Whse\_C climbs steadily, Whse\_S reverses sharply after mid-2015. No single story fits all four.



## Product-level data is noisier than warehouse-level

Residual std as a share of each series' own variability runs higher for products on average (73%) than warehouses (68%) — though not for every single pair.



# 04

## ML Algorithms

*May Zin Ko — ARIMA / ARMA · Random Forest*

# Why ARIMA & Random Forest?

Given only 60 monthly data points per warehouse, models requiring large datasets are de-prioritized. ARIMA and Random Forest offer the best balance of accuracy, interpretability, and data-size fit — and they fail in different, complementary ways.

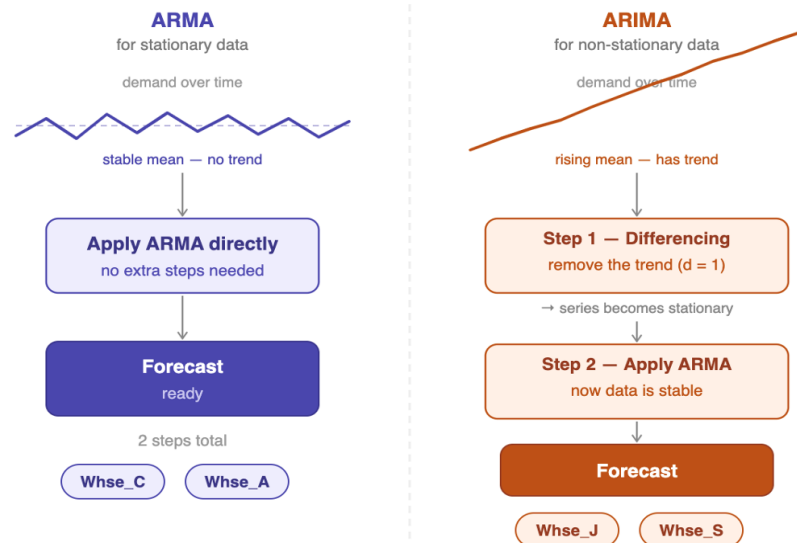
## ARIMA — Classical Statistical

- Strong fit for small time series (as few as 36–48 points)
- Explicitly models trend (d) and autocorrelation (p, q)
- Highly interpretable — every parameter has a clear meaning
- Industry-standard baseline for demand forecasting

## Random Forest — Machine Learning

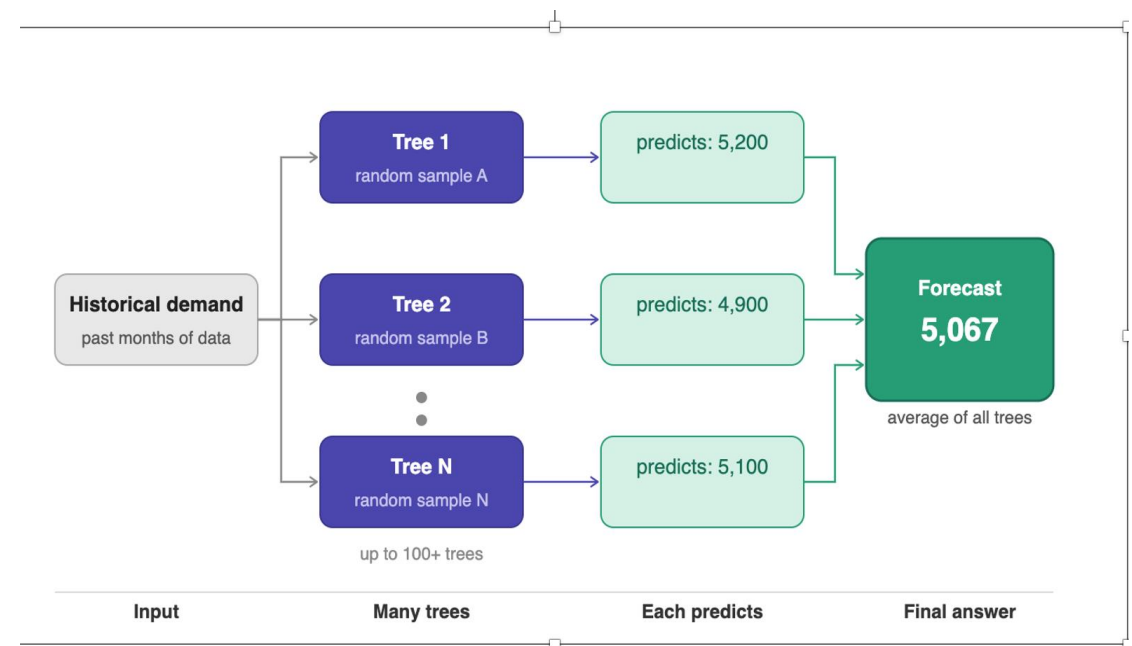
- Captures non-linear patterns ARIMA cannot represent
- Combines data across all 4 warehouses, easing small-sample limits
- Lag + rolling features let it learn seasonal structure empirically
- A meaningfully different model family for honest comparison

### ARMA vs ARIMA



The "I" in ARIMA = Integrated = differencing step

May Zin Ko Visual designed with AI assistance: Anthropic. (2026). Claude (claude-sonnet-4-6) [Large language model]. <https://www.anthropic.com>



# ARIMA(p,d,q) / ARMA — Model Description

*AutoRegressive Integrated Moving Average — one of the most widely used classical methods for time-series forecasting. When  $d = 0$ , the model reduces to ARMA, applied to already-stationary series.*

**p**

## AutoRegressive order

How many past values the model uses to predict the next value

**d**

## Differencing order

How many times the series is differenced to achieve stationarity

**q**

## Moving Average order

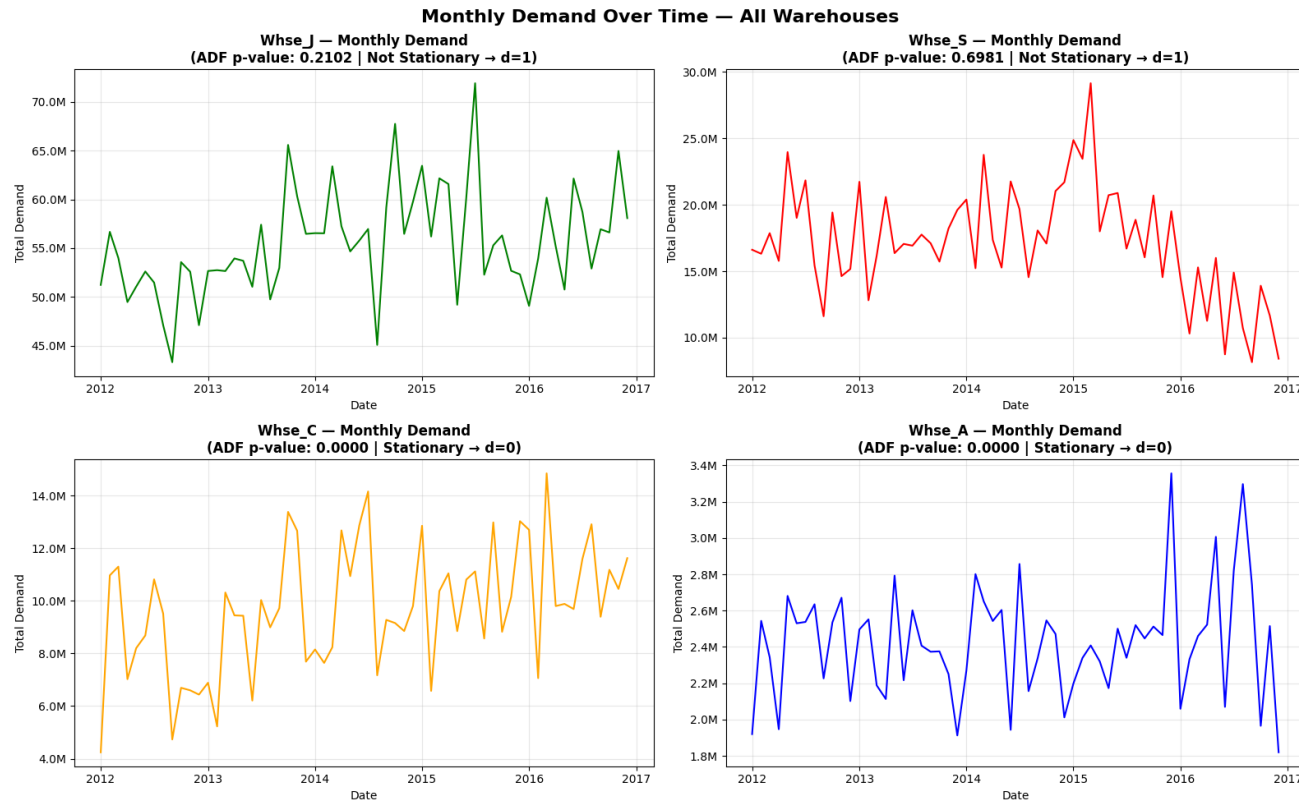
How many past forecast errors are used to correct the prediction

$ARIMA(p, d, q)$

$$y_t' = c + \phi_1 y_{t-1}' + \dots + \phi_p y_{t-p}' + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}'$$

( $y_t'$  = differenced series;  $\varepsilon_t$  = forecast error at time  $t$ )

# ARIMA — ADF (Augmented Dickey-Fuller test)



## Step 1: Test for Stationarity (ADF)

Whse\_J Not Stationary  $\rightarrow d=1$

Whse\_S Not Stationary  $\rightarrow d=1$

Whse\_C Stationary  $\rightarrow d=0$

Whse\_A Stationary  $\rightarrow d=0$

## Step 2: Fit per Warehouse & Product

Each of the 4 warehouses and top-4 products gets its own ARIMA(p,d,q) model — fit independently with d fixed by the ADF result above, and (p, q) chosen via the order-selection process detailed under Improvements.

*8 independent ARIMA models in total: 4 warehouse-level + 4 product-level, each trained on 2012–2015 and tested on 2016.*

# Finding Best $p, q$ — via AIC (auto\_arima)

auto\_arima (pmdarima) automatically searches  $(p, q)$  by minimising AIC, a metric balancing fit against complexity (Liu, 2022, pp. 288–289).  $d$  is fixed per warehouse from the ADF result, not searched — AIC/BIC cannot select differencing order.

```
best = auto_arima(
    train, d=cfg['d'],
    start_p=0, max_p=4, start_q=0, max_q=4,
    information_criterion='aic',
    seasonal=False, stepwise=True,
    suppress_warnings=True
)
# top-3 by MAE on 2016 holdout, best kept
```

Best auto\_arima Order per Warehouse

| Warehouse | Order   | AIC    | MAE       | MAPE  | RMSE      |
|-----------|---------|--------|-----------|-------|-----------|
| Whse_J    | (1,1,1) | 1614.9 | 3,158,711 | 5.5%  | 4,069,946 |
| Whse_S    | (1,1,2) | 1567.5 | 4,564,468 | 45.2% | 5,220,479 |
| Whse_C    | (1,0,2) | 1556.0 | 1,538,425 | 14.2% | 1,978,984 |
| Whse_A    | (0,0,1) | 1342.0 | 342,609   | 14.0% | 421,606   |

*auto\_arima found a different model for every warehouse — no single  $(p,q)$  generalizes. Whse\_S remains the hardest series to fit (45.2% MAPE) regardless of method, reflecting its sharp untrained 2016 decline.*

# Finding Best p, q — via ACF, PACF

Liu (2022, pp. 279–280) explains p is read from the Partial Autocorrelation Function (PACF) cutoff, q from the Autocorrelation Function (ACF) cutoff. PACF isolates the lag's direct effect after removing what shorter lags already explain.

```
threshold = 2 / sqrt(len(ts_plot))
pacf_vals = pacf(ts_plot, nlags=20)
acf_vals = acf(ts_plot, nlags=20)

# p = first lag where |PACF| > threshold
# q = first lag where |ACF| > threshold
model = ARIMA(train, order=(p, d, q)).fit()
```

## ACF/PACF Suggested Order per Warehouse

| Warehouse | Order   | MAE       | MAPE  | RMSE      |
|-----------|---------|-----------|-------|-----------|
| Whse_J    | (1,1,1) | 4,043,529 | 6.9%  | 4,981,195 |
| Whse_S    | (1,1,1) | 5,944,462 | 57.9% | 6,548,456 |
| Whse_C    | (4,0,0) | 1,946,132 | 17.1% | 2,392,709 |
| Whse_A    | (4,0,0) | 355,698   | 14.5% | 424,096   |

*ACF/PACF underperforms across every warehouse here — it works best on clean textbook series with sharp cutoffs. Aggregated, noisy warehouse demand doesn't produce a clean cutoff, making visual threshold reading unreliable.*



# Finding Best p, q — Manual Grid Search

An exhaustive search over  $p \in [0,4]$ ,  $d \in [0,2]$ ,  $q \in [0,4]$ , each combination evaluated by MAE directly on the 2016 holdout. Manual grid search avoids the cross-validation-on-time-series validity problem by always testing on the true future holdout.

```
for p in range(0, max_p+1):
    for d in range(0, max_d+1):
        for q in range(0, max_q+1):
            model = ARIMA(train, order=(p,d,q)).fit()
            forecast = model.forecast(steps=len(test))
            mae = mean_absolute_error(test, forecast)
# select lowest MAE on 2016 holdout
```

## Best Manual Order per Warehouse

| Warehouse | Order   | MAE       | MAPE  | RMSE      |
|-----------|---------|-----------|-------|-----------|
| Whse_J    | (0,2,2) | 3,174,299 | 5.6%  | 3,815,396 |
| Whse_S    | (0,2,3) | 5,245,640 | 51.5% | 5,871,384 |
| Whse_C    | (1,2,4) | 1,426,463 | 13.6% | 1,807,621 |
| Whse_A    | (2,0,1) | 337,969   | 13.7% | 421,752   |

*Manual search wins for Whse\_C and Whse\_A — its higher-order/higher-differencing combinations (e.g. ARIMA(1,2,4)) capture patterns auto\_arima's AIC penalty steers away from. This shows AIC-based selection isn't always sufficient.*

# ARIMA — Order Selection: Comparing 3 Methods

```
def find_best_arma(train, test, max_p=4, max_d=2, max_q=4):
    results = {}
    for p in range(max_p+1):
        for d in range(max_d+1):
            for q in range(max_q+1):
                model = ARIMA(train, order=(p,d,q)).fit()
                forecast = model.forecast(steps=len(test))
                mae = mean_absolute_error(test, forecast)
                results[(p,d,q)] = mae
    # select lowest MAE on 2016 holdout
```

Manual grid search avoids the cross-validation-on-time-series validity problem by always testing on the true future holdout.

AIC: Akaike Information Criterion  
ACF: Auto Correlation Function  
PACF: Partial Auto Correlation Function

| Method             | Order        | MAPE |
|--------------------|--------------|------|
| Manual Grid Search | ARIMA(0,2,2) | 5.6% |
| auto_arma (AIC)    | ARIMA(1,1,1) | 5.5% |
| ACF / PACF         | ARIMA(1,1,1) | 6.9% |

(Whse\_J shown above) — auto\_arma won here; manual grid search won on others. There is no single best method across every series.

## Final Method Selected per Series

Whse\_J: auto\_arma

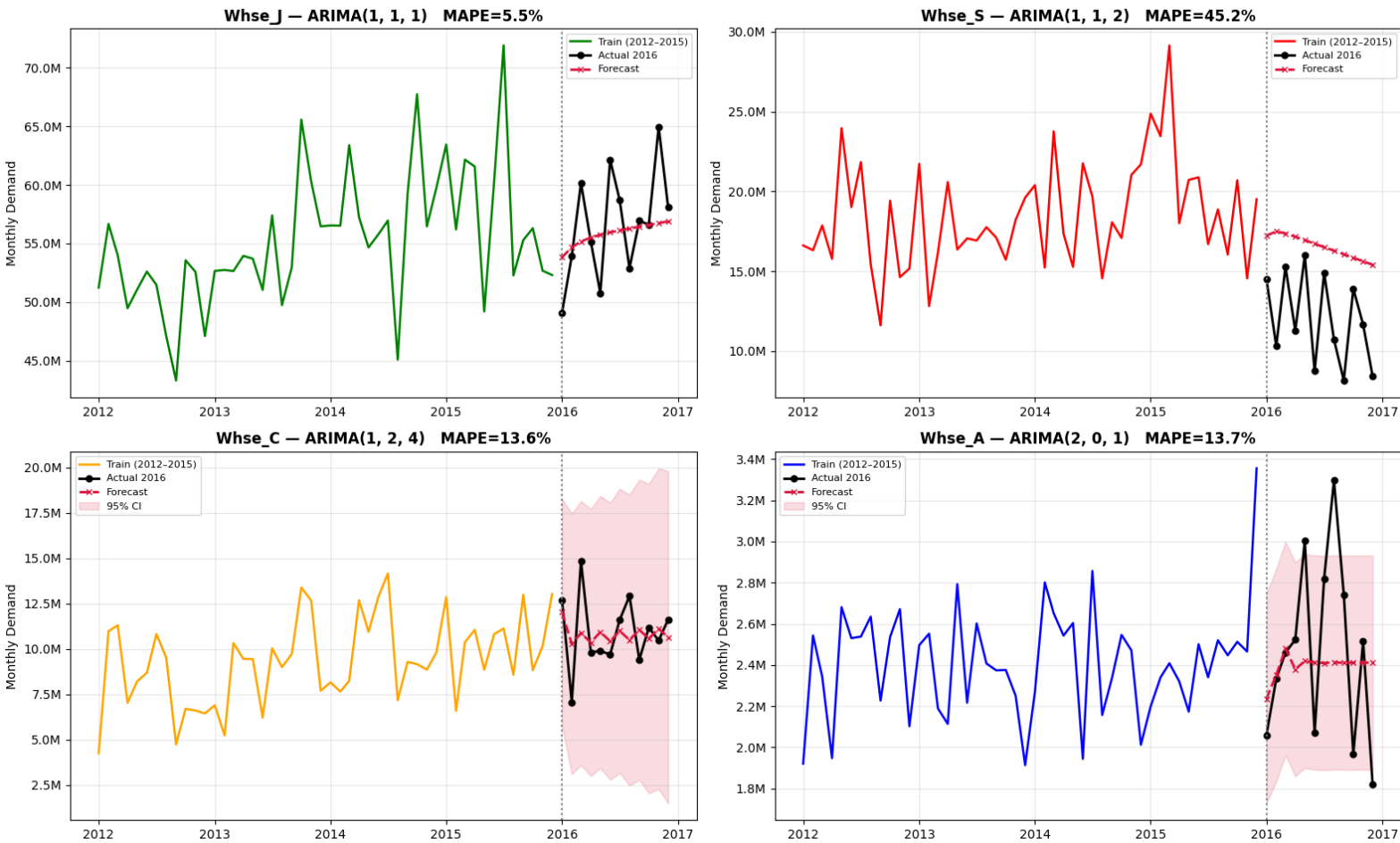
Whse\_S: manual / auto\_arma

Whse\_C: manual

Whse\_A: manual

# ARIMA — Performance by Warehouse

ARMA / ARIMA Forecast vs Actual by Warehouse (2016) — Best Orders



Whse\_J ARIMA(1,1,1)

MAPE 5.5%

Whse\_S ARIMA(1,1,2)

MAPE 45.2%

Whse\_C ARIMA(1,2,4)

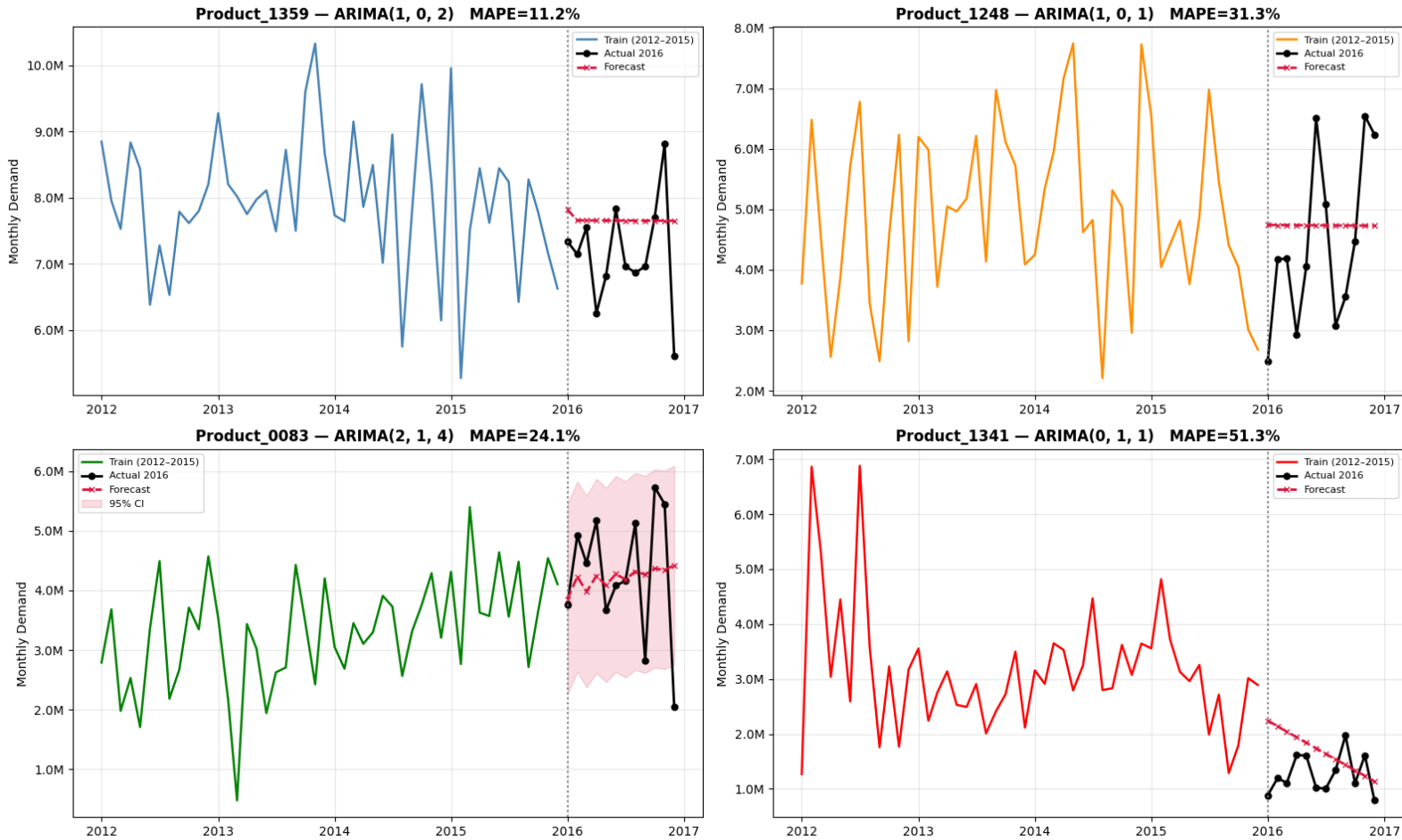
MAPE 13.6%

Whse\_A ARIMA(2,0,1)

MAPE 13.7%

# ARIMA — Final Results by Product

ARIMA Forecast vs Actual — Top 4 Products (2016)



## Product\_1359

ARIMA(1,0,2) · MAPE 11.2%

## Product\_1248

ARIMA(1,0,1) · MAPE 31.3%

## Product\_0083

ARIMA(2,1,4) · MAPE 24.1%

## Product\_1341

ARIMA(0,1,1) · MAPE 51.3%

# What is Random Forest?

*An ensemble of decision trees, each trained on a random subset of data and features; predictions are averaged across all trees. Unlike ARIMA, which models time dependencies directly, Random Forest treats forecasting as a supervised regression problem learned from engineered features.*

## Why Random Forest for this dataset?

- Handles non-linear relationships that ARIMA cannot capture
- No stationarity requirement — no ADF test or differencing needed
- Naturally handles multiple input features simultaneously
- Robust to outliers due to ensemble averaging
- Captures trend patterns through lag and rolling-mean features
- Handles all 4 warehouses (or all 4 products) in one combined model using an encoded warehouse\_code / product\_code feature

### n\_estimators

= 100

Number of trees in the forest

### random\_state

= 42

Seed for reproducibility

*ARIMA models the time series directly. Random Forest requires the series to be transformed into a feature matrix first — each row is one month, with columns for lag values, calendar features, and rolling averages. This transformation, feature engineering, is the most critical step.*

# Random Forest — Model Design

```
# Combined-warehouse training (not per-warehouse)
X_train = df_train_all.drop(columns='demand')
y_train = df_train_all['demand']

model = RandomForestRegressor(
    n_estimators=100, random_state=42
)
model.fit(X_train, y_train)  # one model, all 4 warehouses

# Evaluate per warehouse on 2016 holdout
forecast = model.predict(X_test_wh)
```

## Why combine all 4 warehouses into one model?

Per-warehouse training has only ~36 rows — too little for tree splitting. The combined model pools ~144 rows and learns shared seasonal patterns across all warehouses simultaneously, which is why this approach was chosen over training separately.

## Two Scopes, Two Models

### Warehouse Model

1 combined model across all 4 warehouses, ~144 training rows

### Top-4 Product Model

1 combined model across the 4 top products, smaller & more volatile

*Direct forecasting (not recursive) was chosen for Random Forest after empirical comparison — it produced lower error on this dataset.*

# Random Forest — Model Description

*An ensemble of decision trees, each trained on a random subset of data and features; predictions are averaged across trees. Not natively time-aware, so past values must be manually engineered as lag and rolling-mean features.*

## **n\_estimators**

**Number of trees in the forest**

More trees → more stable predictions, higher compute cost

## **max\_depth**

**Maximum depth of each tree**

Limits how complex a single tree can grow; controls overfitting

## **min\_samples\_leaf**

**Minimum samples required at a leaf node**

Larger values smooth predictions, reduce overfitting on noisy data

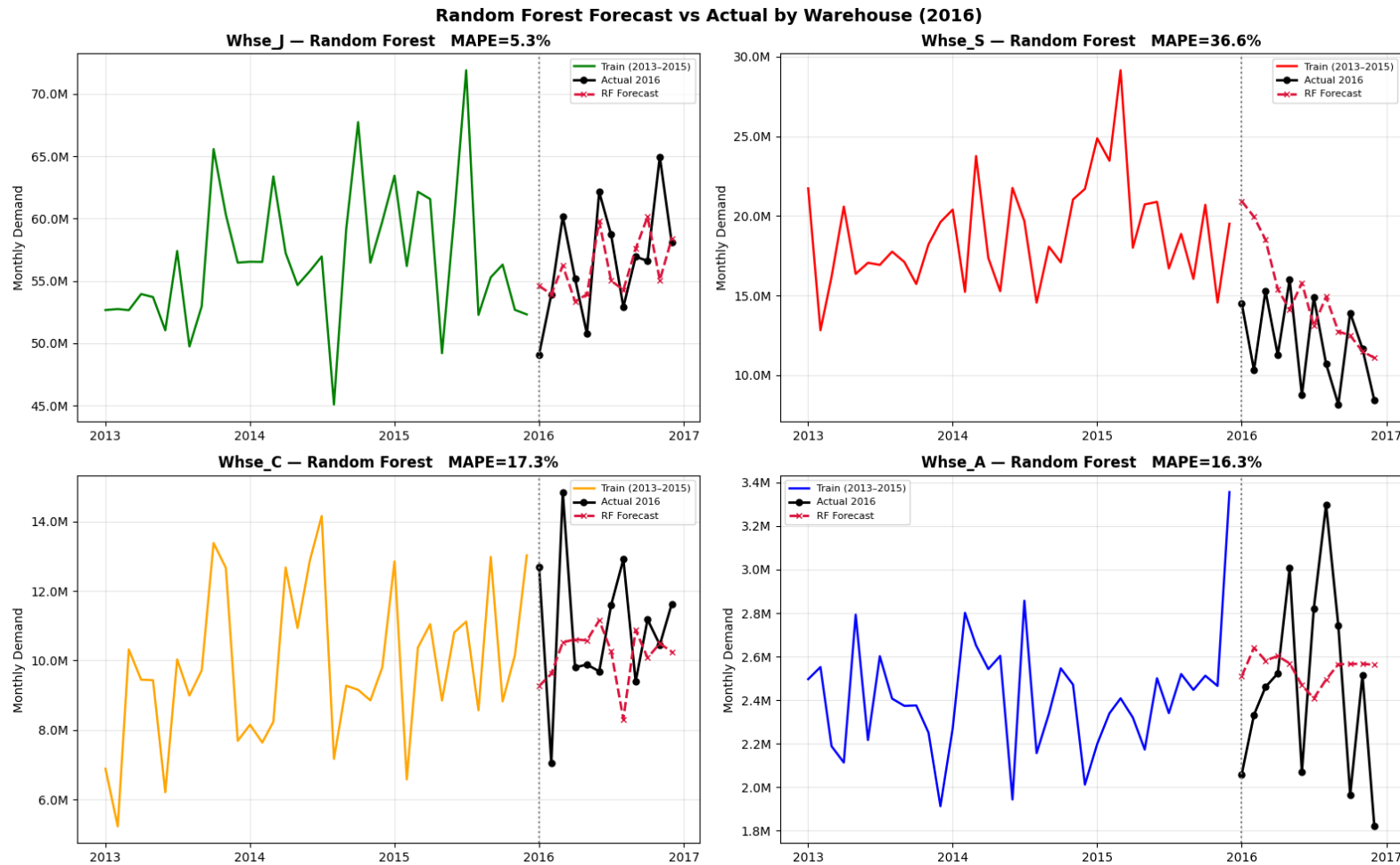
## **random\_state**

**Seed for reproducibility**

Ensures the same random subsets are used across runs

*Because Random Forest has no built-in concept of time, every “temporal” signal it sees — trend, seasonality, recency — has to be engineered as an explicit input feature.*

# Random Forest — Performance by Warehouse



Whse\_J

MAPE 5.3%

Whse\_S

MAPE 36.6%

Whse\_C

MAPE 17.3%

Whse\_A

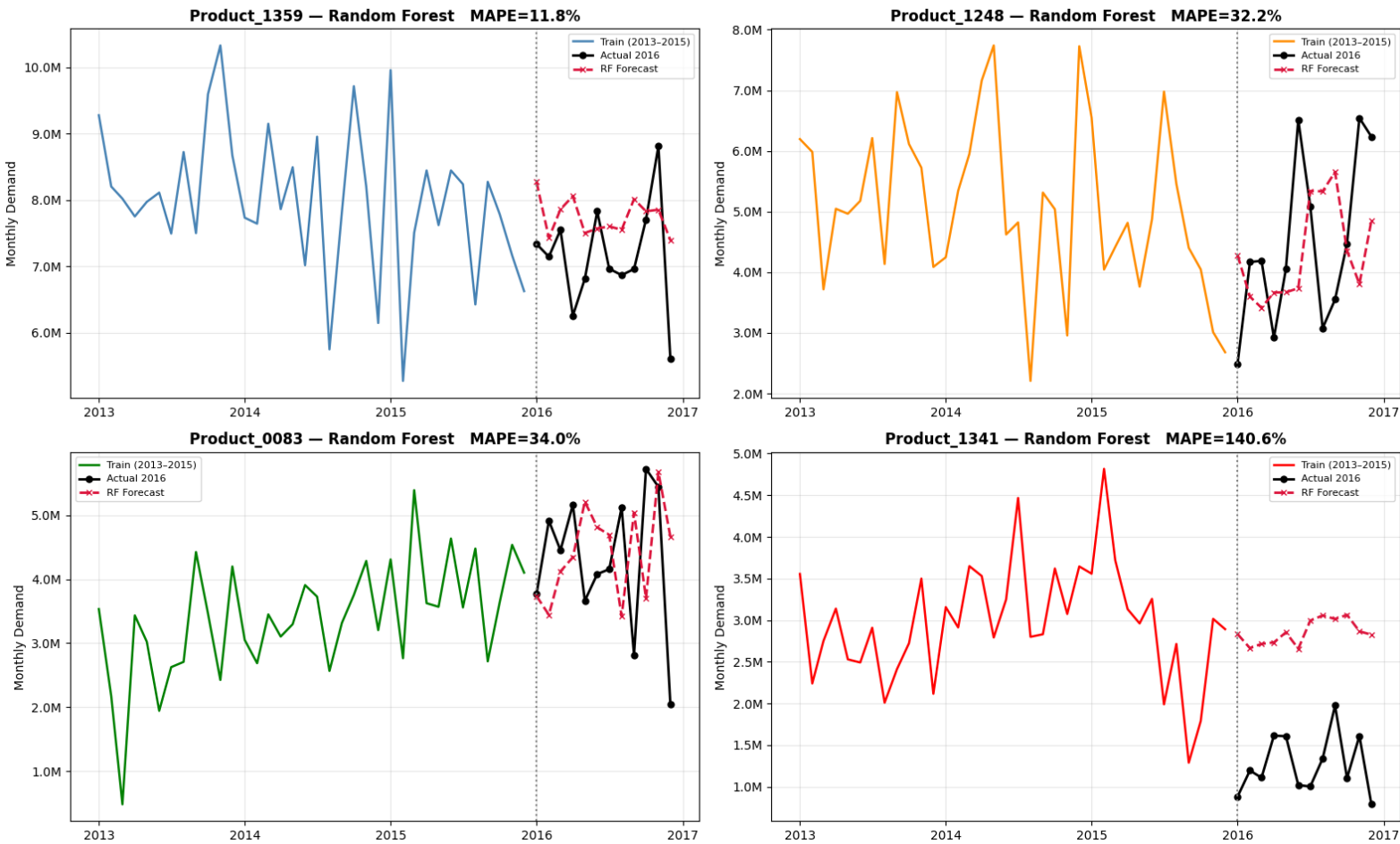
MAPE 16.3%

Combined-warehouse training, baseline config ( $n_{estimators}=100$ , default depth).



# Random Forest — Final Results by Product

Random Forest Forecast vs Actual — Top 4 Products (2016)



Product\_1359

MAPE 11.8%

Product\_1248

MAPE 32.2%

Product\_0083

MAPE 34.0%

Product\_1341

MAPE 140.6%

Product\_1341's 140.6% MAPE is the single worst result in this entire project — a key theme in the Final Performance Summary next.

# Random Forest — Hyperparameter Tuning

```
param_grid = [  
    {'n_estimators':100,'max_depth':None,'min_samples_leaf':1}, # baseline  
    {'n_estimators':200,'max_depth':None,'min_samples_leaf':1},  
    {'n_estimators':100,'max_depth':5, 'min_samples_leaf':1},  
    {'n_estimators':100,'max_depth':10, 'min_samples_leaf':1},  
    {'n_estimators':200,'max_depth':10, 'min_samples_leaf':2},  
    {'n_estimators':300,'max_depth':10, 'min_samples_leaf':2},  
]  
# Manual grid search, train 2012-15 / test 2016 (no k-fold CV)
```

## Warehouse Model — Tuning Results

| Config                       | MAPE    |
|------------------------------|---------|
| n=100, depth=None (baseline) | 18.9% ★ |
| n=100, depth=5               | 18.9%   |
| n=200, depth=10, leaf=2      | 20.1%   |

### Warehouse-level finding

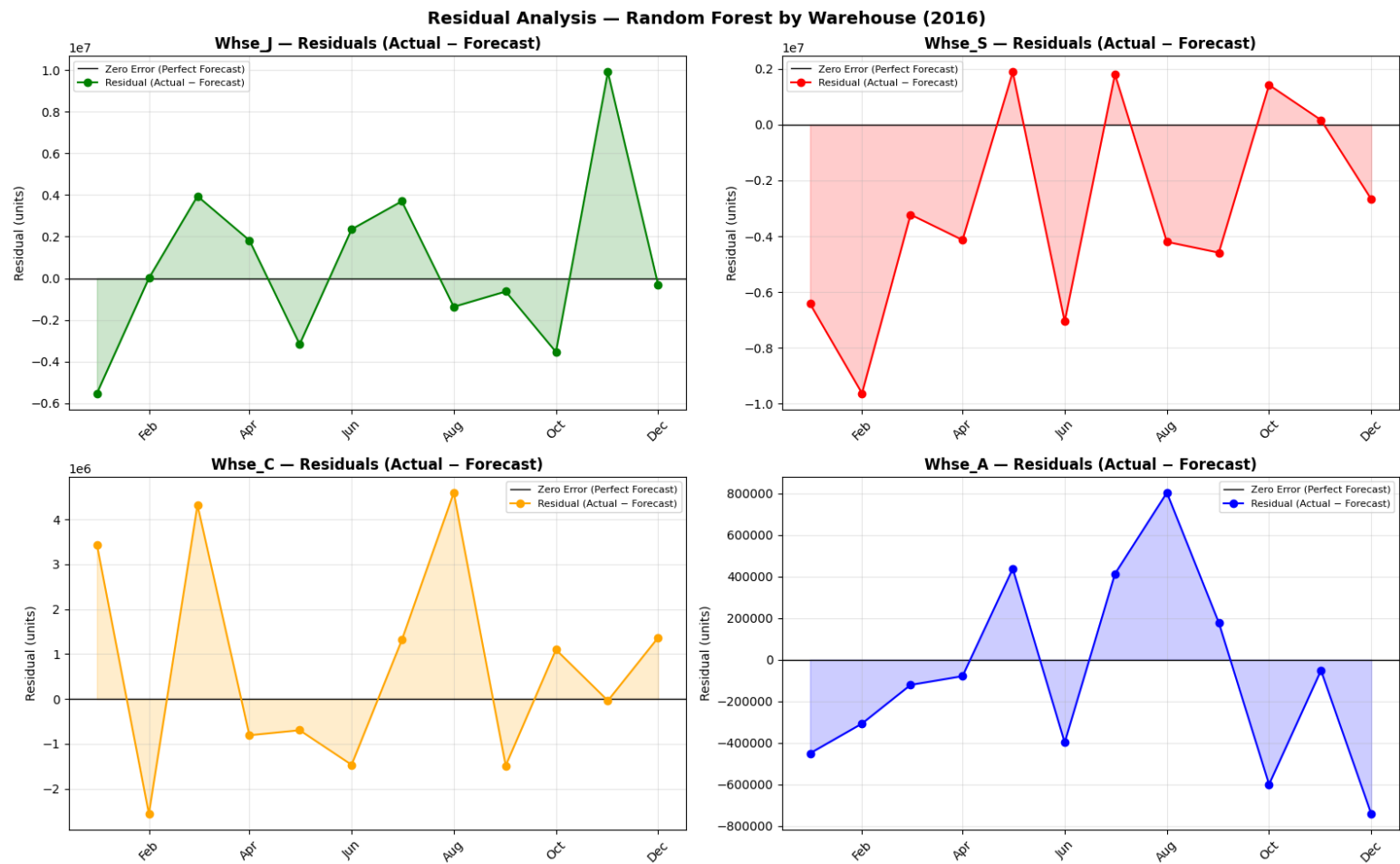
The default baseline configuration outperformed every tuned variant. The ~19% error ceiling is set by feature availability, not model configuration.

### Top-4 Product model finding

n\_estimators=200, max\_depth=10, min\_samples\_leaf=2 reduced average MAPE from 54.6% → 52.5%. The smaller, more volatile product-level data benefited from extra regularization.

*A “failed” tuning result is still useful — it shows the error ceiling comes from feature availability, pointing toward feature engineering as the next lever.*

# Residual / Error Analysis — Diagnosing Remaining Error

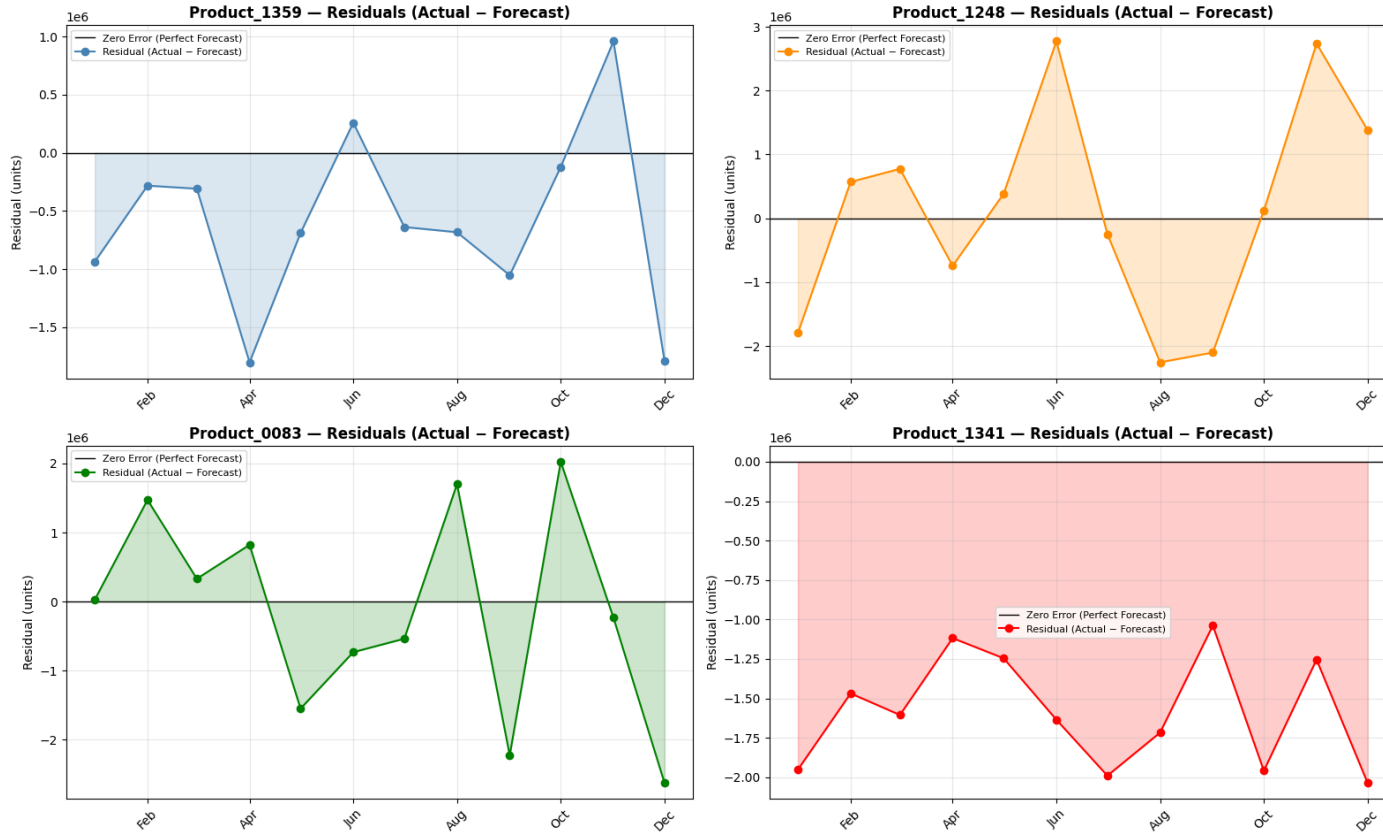


- Whse\_S residuals are almost entirely negative in 2016 — a systematic over-forecasting bias, not random noise
- Whse\_C and Whse\_A residuals are balanced in both directions — closer to genuine random error
- At the product level, Product\_1341 shows the same one-directional bias — RF cannot follow its structural 2016 demand collapse

*A systematic bias can be corrected with an adjustment factor; balanced noise cannot — this distinction shapes the recommendations ahead.*

# Residual / Error Analysis

Residual Analysis — Random Forest by Product (2016)



- Product\_1341 — mean residual  $-1.59M$ , by far the most negative and the lowest std (344K): a strongly systematic, consistent over-forecast — RF cannot follow the demand collapse below its training range.
- Product\_1359 — mean  $-591K$ , worst month Apr 2016; mild over-forecasting bias.
- Product\_1248 — mean  $+133K$  but the largest spread (std  $1.62M$ ) — errors are large but balanced in both directions.
- Product\_0083 — mean  $-126K$ , worst month Dec 2016, close to balanced random error.

*Product\_1341 shows the same one-directional bias seen at the warehouse level (Whse\_S) — RF cannot follow a structural 2016 demand collapse.*

# Final Performance Summary — ARIMA vs. Random Forest

By Warehouse

| Warehouse | ARIMA | RF    |
|-----------|-------|-------|
| Whse_J    | 5.5%  | 5.3%  |
| Whse_S    | 45.2% | 36.6% |
| Whse_C    | 13.6% | 17.3% |
| Whse_A    | 13.7% | 16.3% |

By Top-4 Product

| Product      | ARIMA | RF     |
|--------------|-------|--------|
| Product_1359 | 11.2% | 11.8%  |
| Product_1248 | 31.3% | 32.2%  |
| Product_0083 | 24.1% | 34.0%  |
| Product_1341 | 51.3% | 140.6% |

## Headline Finding

- Random Forest edges out ARIMA at warehouse level for Whse\_J and Whse\_S; ARIMA wins for Whse\_C and Whse\_A
- ARIMA wins on every single product — it handles trend and structural change far better than Random Forest at this granularity
- Random Forest's inability to extrapolate beyond its training range (Product\_1341: 140.6%) is the single most important finding of this comparison
- This sets up a hybrid recommendation, detailed fully in the Model Comparison chapter ahead

## 05

# ML Algorithms

*Nashitah — SARIMA · XGBoost · Prophet*

# SARIMA, XGBoost & Prophet — Model Descriptions

## SARIMA

### Seasonal ARIMA(p,d,q)(P,D,Q,12)

Extends ARIMA with explicit seasonal autoregressive, differencing and moving-average terms at a 12-month period — built for monthly demand with recurring annual structure.

## XGBoost

### Gradient-boosted decision trees

Learns from engineered features — lag demand, month, year, rolling averages. Useful when demand is nonlinear and not well described by a fixed seasonal formula.

## Prophet

### Additive trend + seasonality model

Designed for business time series. Fast and automated, with minimal manual tuning — convenient when forecasting must be repeated across many series.

# Why SARIMA, XGBoost & Prophet?

Three models with different strengths, evaluated against the same naive (10.3%) and 3-month moving-average (8.4%) MAPE baseline established on Product\_1359 / Whse\_J.

## SARIMA — Classical

- Explicitly models the 12-month seasonal cycle ARIMA leaves implicit
- Interpretable seasonal + trend parameters
- Natural extension once ARIMA showed some series need explicit seasonality

## XGBoost — Machine Learning

- Captures nonlinear, irregular demand patterns
- Lag + calendar features let it learn structure empirically
- A meaningfully different model family from both ARIMA and SARIMA

## Prophet — Business-Oriented

- Fast, automated baseline across many warehouse/product series
- Useful for comparing which series are predictable in practice
- Minimal manual tuning — good fit for a repeated-forecast workflow



# Setup, Baseline & Train/Test Split

```
monthly = df.groupby([
    'Warehouse', 'Product_Category',
    pd.Grouper(key='Date', freq='MS')
])['Order_Demand'].sum()

train = monthly[date.year <= 2015]
test  = monthly[date.year == 2016]
```

## Baseline to Beat (Product\_1359 / Whse\_J)

| Baseline           | MAE     | RMSE    | MAPE  |
|--------------------|---------|---------|-------|
| Naive              | 759,750 | 936,669 | 10.3% |
| 3-Month Moving Avg | 584,417 | 774,786 | 8.4%  |

The moving-average baseline beats naive on every metric, so SARIMA and XGBoost must clear about 8.4% MAPE on this series to be useful.

### Same methodology across all three models

Monthly aggregation by Warehouse × Product\_Category, trained on 2012–2015, tested on the 2016 holdout — identical to the train/test split used for ARIMA and Random Forest, so results are directly comparable in the Model Comparison chapter.

# Why Train/Test — Not Train/Validation/Test?

*This was a deliberate methodological choice, not an oversight — here is the reasoning:*

## 1. Time series data cannot be randomly split

A validation set in time series is simply another fixed time window cut from already-limited training data. With only 48 monthly points (2012–2015) per series, a 3-way split would leave as few as 24–30 training months — too few for SARIMA, XGBoost, or Prophet to learn meaningful patterns.

## 2. Each model has a built-in selection mechanism

ARIMA and SARIMA select order via AIC — an information criterion that replaces the need for a held-out validation set (Liu, 2022, pp. 288–290). XGBoost used early stopping on the test set. Prophet fits trend and seasonality components automatically. None of the five models rely on a separate validation window.

## 3. ML models used the test set for hyperparameter search

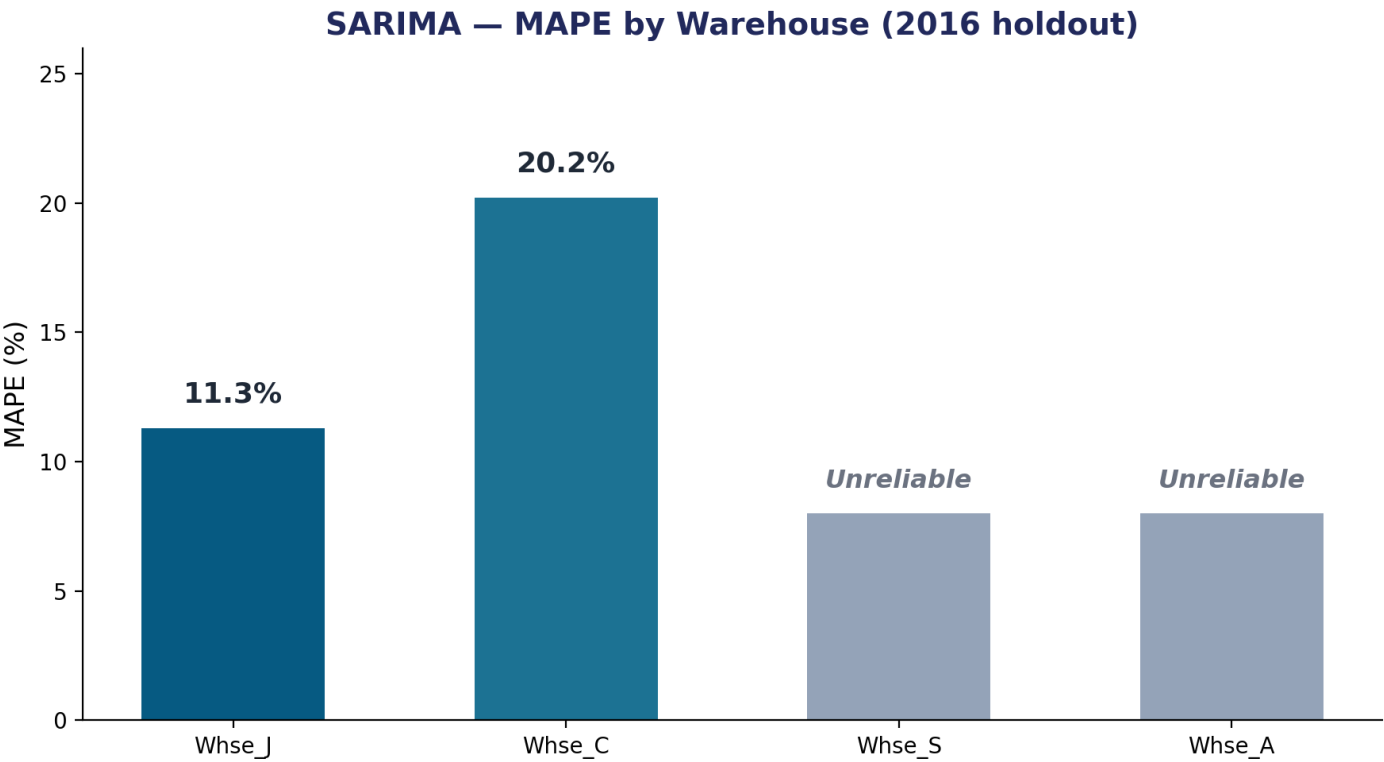
Random Forest and XGBoost hyperparameters were evaluated directly on the 2016 holdout. For time series this is appropriate — k-fold cross-validation would shuffle time order, breaking the sequential structure that all models depend on.

## 4. Train/test is accepted practice for short time series

Standard forecasting literature (e.g. Hyndman & Athanasopoulos, 2021) recommends a single holdout window for evaluation when data is limited — the same approach used here for all five models, enabling fair comparison.

**All five models use the same split: 2012–2015 training / 2016 holdout test — ensuring results are directly comparable across ARIMA, Random Forest, SARIMA, XGBoost, and Prophet.**

# SARIMA — Performance by Warehouse



Same warehouse × category aggregation as ARIMA (Category\_028 for J/S/A, Category\_019 for C), same 2012–2015 / 2016 split.

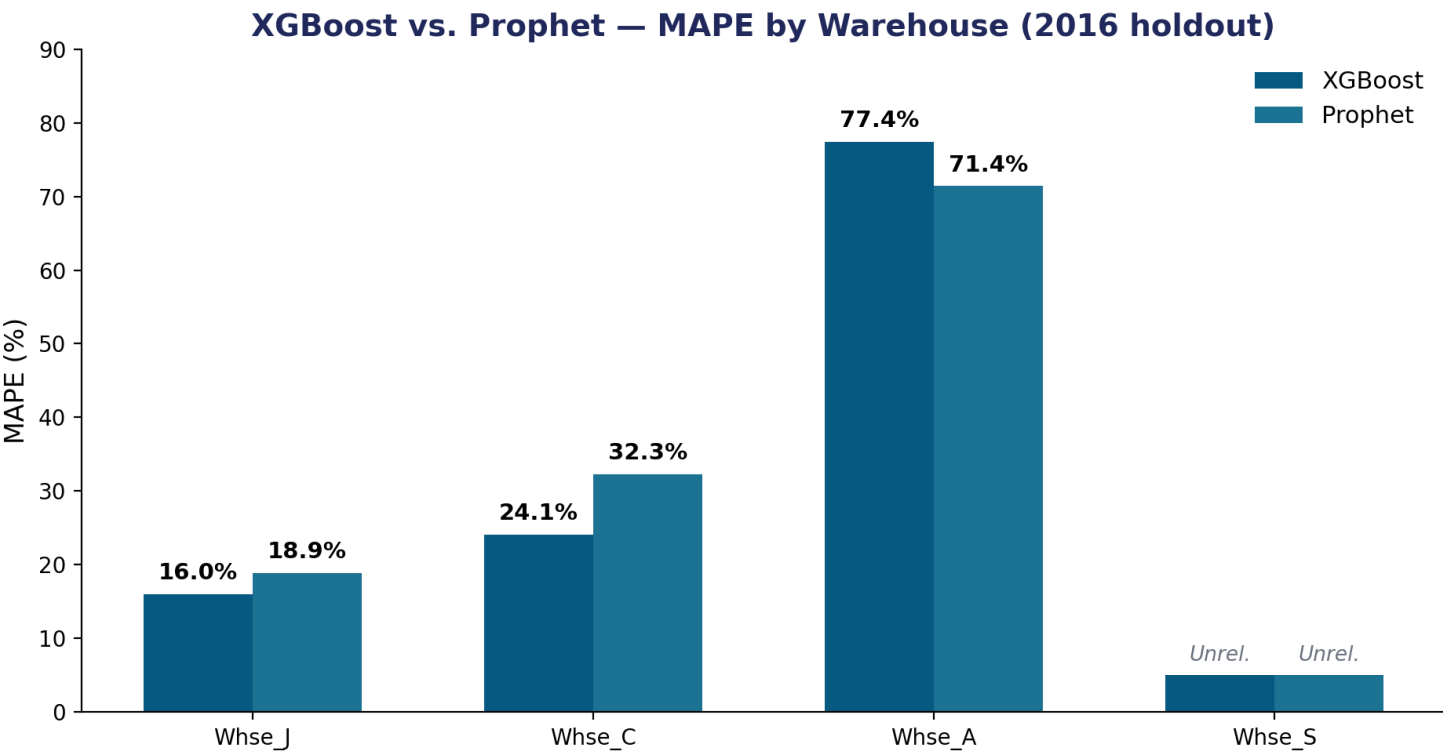
## Findings

Whse\_J (11.3%) and Whse\_C (20.2%) are usable for planning.

**Whse\_S & Whse\_A: Unreliable**

MAPE explodes to 4,569% and 298% — near-zero actual-demand months distort the ratio. MAE/RMSE remain sane; the percentage metric breaks down, not the forecast itself.

# XGBoost & Prophet — Performance by Warehouse



## Findings

Whse\_J is XGBoost's strongest case (16.0%); Prophet is close behind (18.9%).

Whse\_C: Prophet (32.3%) trails XGBoost (24.1%) — the stable, stationary demand here favors the feature-based model.

Whse\_A: both models struggle (71–77%) — weak but not broken.

Whse\_S unreliable for both (near-zero demand months)

Neither model beats SARIMA's two usable warehouses (J, C) — a pattern that carries into the Model Comparison chapter.

# Extending to Top-4 Products — Validity Check

Same 4 products as May Zin Ko's ARIMA/RF work (Product\_1359, Product\_1248, Product\_0083, Product\_1341). Before modeling, every warehouse pairing is checked for enough history.

1

### Identify Pairs

7 raw product×warehouse pairs found across the dataset (e.g. Product\_0083 spans Whse\_C, Whse\_J, Whse\_S).

2

### Count Monthly Points

Every pair has 60–61 monthly observations — enough history to attempt forecasting.

3

### Flag Usable Pairs

Rule: ≥18 monthly points AND data present in train, validation & test. All 7 pairs qualify.

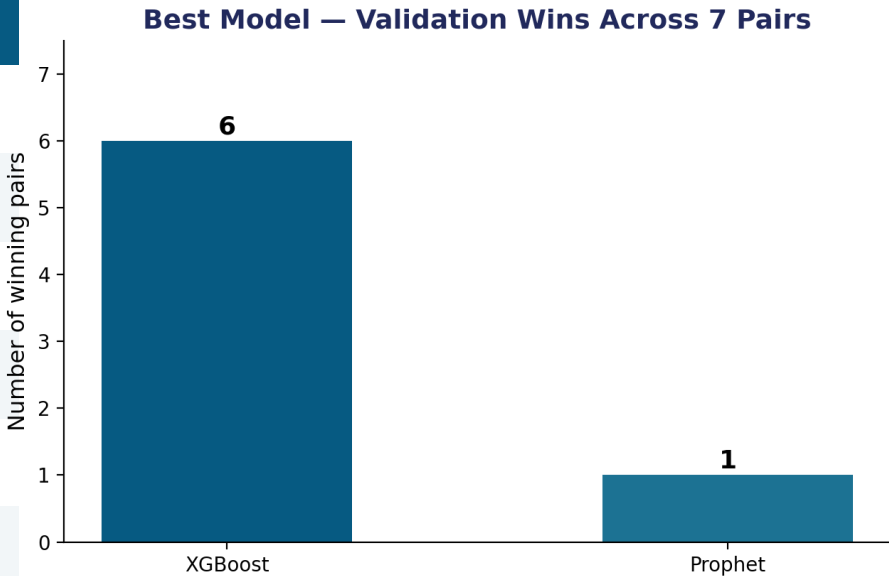
## All 7 Pairs Are Usable

| Product      | Warehouse | Monthly Points | Train Rows | Test Rows |
|--------------|-----------|----------------|------------|-----------|
| Product_0083 | Whse_C    | 60             | 199        | 68        |
| Product_0083 | Whse_J    | 60             | 774        | 259       |
| Product_0083 | Whse_S    | 60             | 226        | 81        |
| Product_1248 | Whse_J    | 60             | 3,944      | 1,176     |
| Product_1341 | Whse_C    | 61             | 489        | 129       |
| Product_1341 | Whse_S    | 61             | 1,309      | 171       |
| Product_1359 | Whse_J    | 61             | 10,238     | 2,805     |

# Best Model Selection — Validation Results

Prophet and XGBoost run on all 7 valid pairs; the better model per pair is chosen mainly on MAPE.

| Pair                  | Best Model | MAPE   |
|-----------------------|------------|--------|
| Product_0083 / Whse_C | XGBoost    | 134.2% |
| Product_0083 / Whse_J | XGBoost    | 30.7%  |
| Product_0083 / Whse_S | Prophet    | 28.3%  |
| Product_1248 / Whse_J | XGBoost    | 33.6%  |
| Product_1341 / Whse_C | XGBoost    | 36.2%  |
| Product_1341 / Whse_S | XGBoost    | 41.5%  |
| Product_1359 / Whse_J | XGBoost    | 18.1%  |

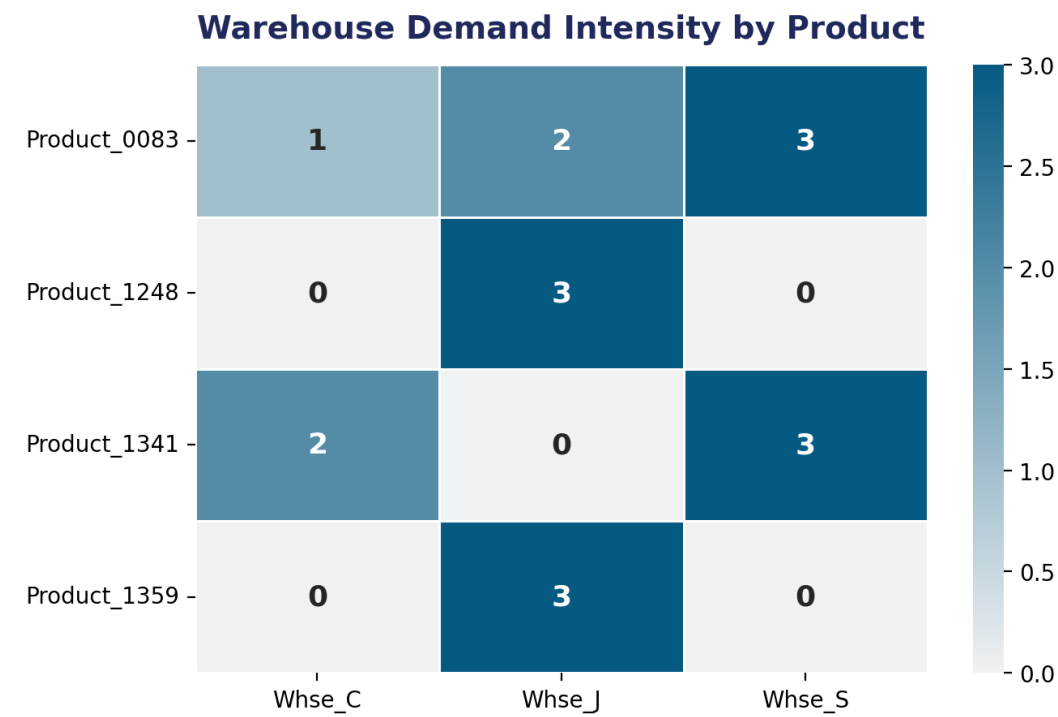


XGBoost wins 6 of 7 pairs; Prophet only for Product\_0083 / Whse\_S.

**Strongest pair: Product\_1359 / Whse\_J (18.1% MAPE). Weakest pair: Product\_0083 / Whse\_C (134.2% MAPE).**

Both products and both models are usable — but trust varies sharply by warehouse, which is exactly what drives the operational recommendations ahead.

# Where Demand Concentrates — Warehouse × Product

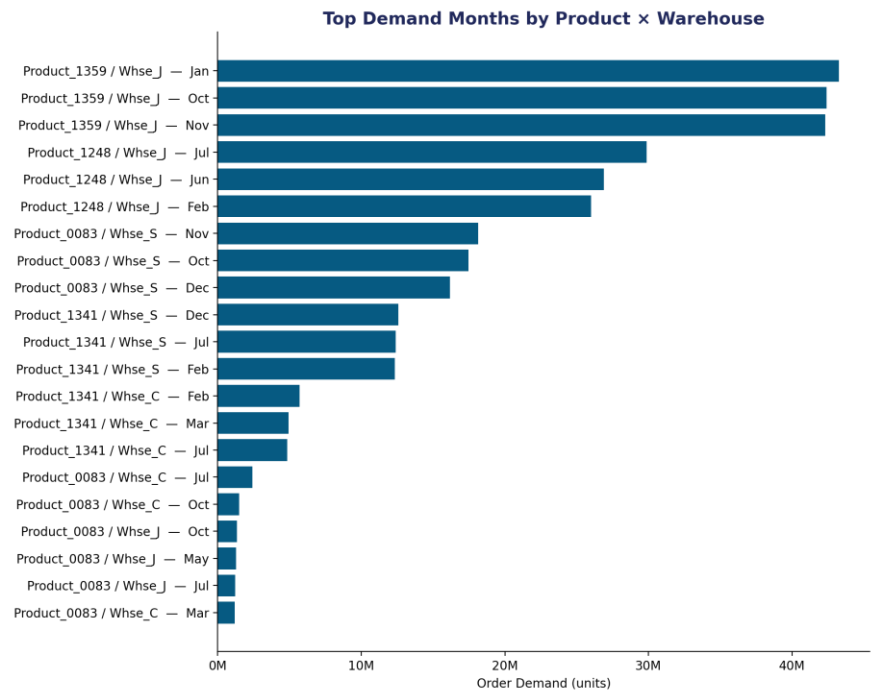


## Business Reading

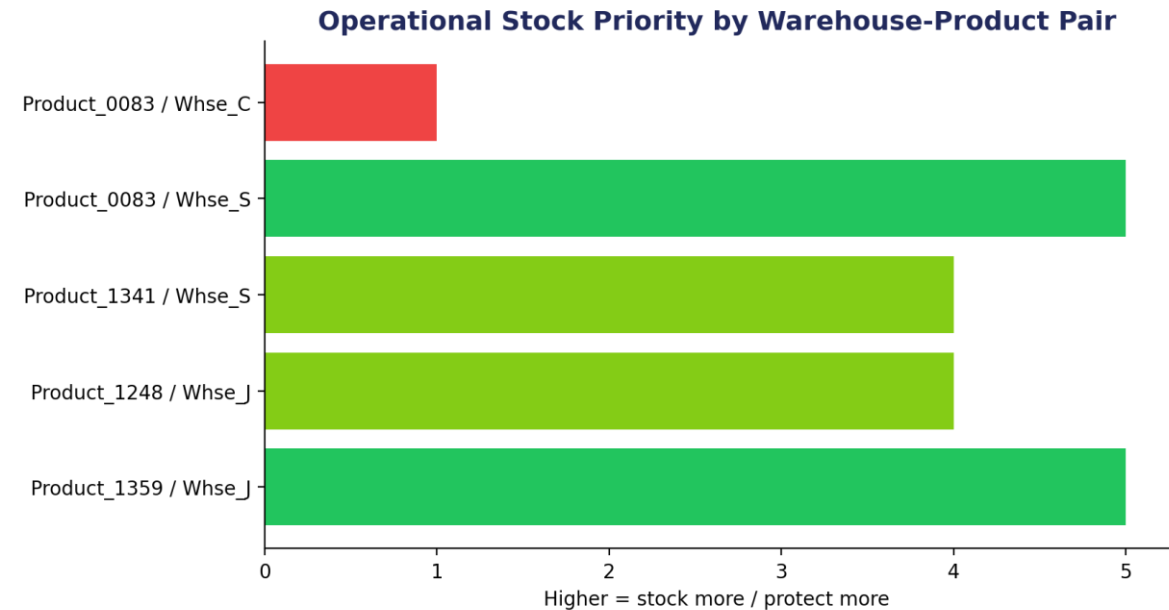
Product\_1359 and Product\_1248 are concentrated almost entirely in Whse\_J.  
Product\_0083 and Product\_1341 are stronger in Whse\_S, with lighter, more irregular presence elsewhere.  
Whse\_C never holds the dominant share for any of the 4 products — consistent with it being the weakest forecast warehouse too.

Demand concentration explains forecast quality: where one warehouse dominates a product, the series is cleaner and easier to forecast (Product\_1359/Whse\_J, 18.1% MAPE).

# Seasonality & Stock Priority — Business Findings



Product\_1359/Whse\_J peaks Jan, Oct, Nov; Product\_1248/Whse\_J peaks Jun–Jul. Product\_1341/Whse\_S peaks sharply in December.



Product\_1359/Whse\_J and Product\_0083/Whse\_S score highest (5) — build stock ahead of their peak months.  
Product\_0083/Whse\_C scores lowest (1) — the weakest, hardest-to-forecast pair; avoid overstock here.

Demand is highly warehouse-specific — inventory strategy must be set per warehouse-product pair, not applied uniformly. Next: comparing all five models together.



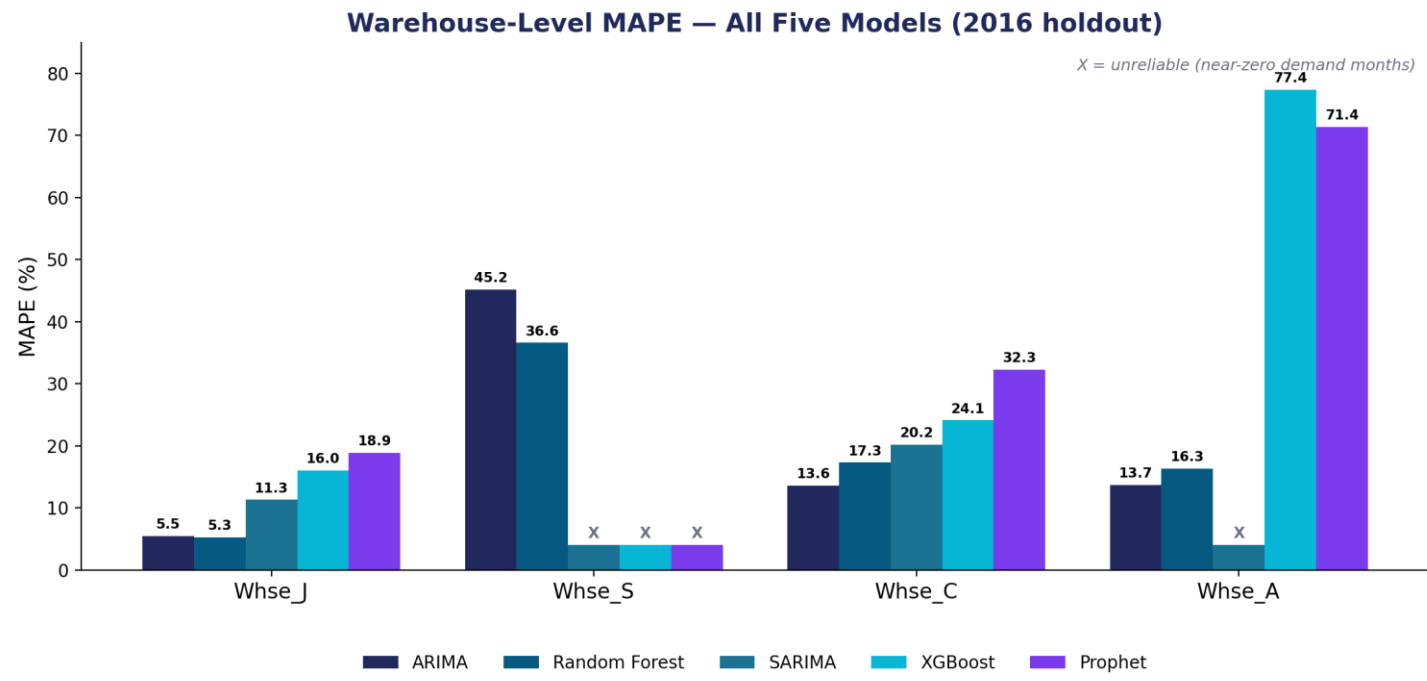
# 06

## Model Comparison

*All Five Models — ARIMA, Random Forest, SARIMA, XGBoost, Prophet*

# Warehouse-Level — All Five Models

2016 holdout MAPE, all five models, side by side.



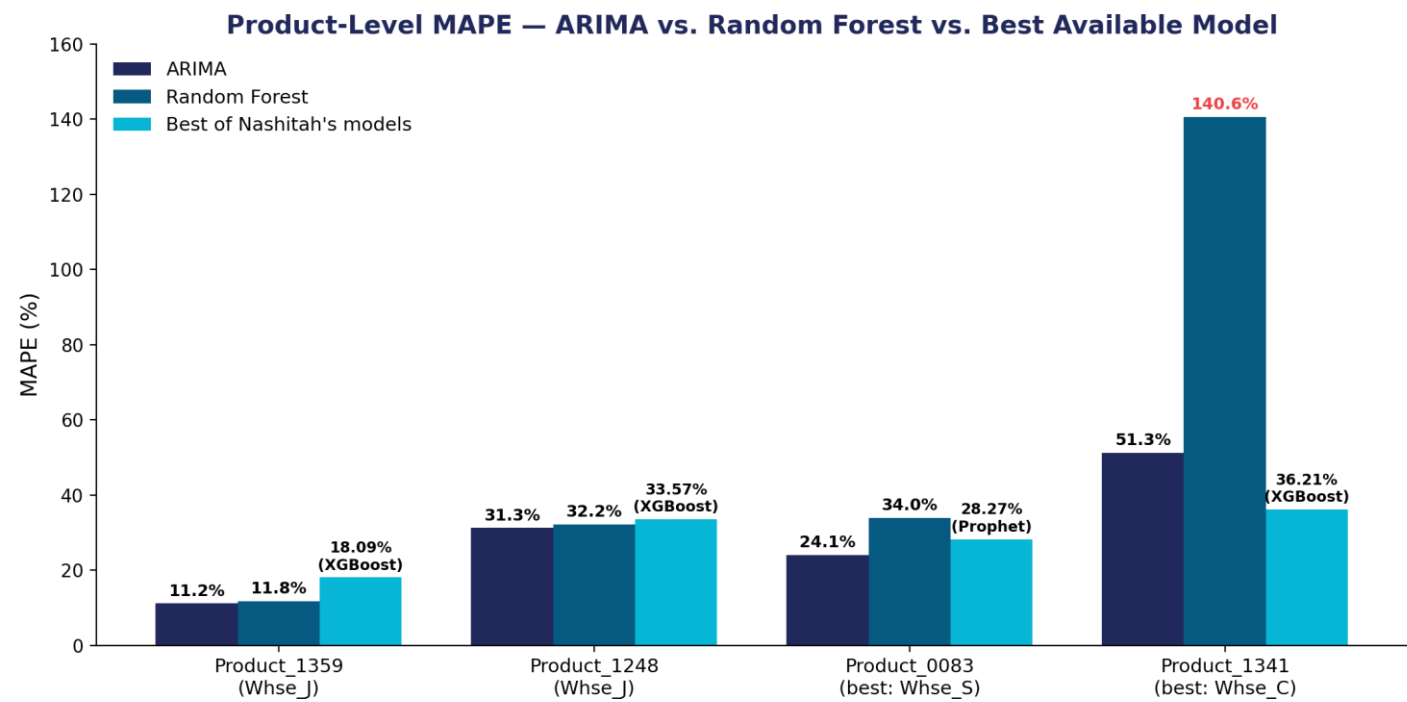
## Adding 3 more models doesn't change the hybrid recommendation

Random Forest still wins Whse\_J & Whse\_S; ARIMA still wins Whse\_C & Whse\_A. SARIMA, XGBoost and Prophet never beat both at any warehouse — and they're the ones that break down (Unreliable) at Whse\_S and, for SARIMA, at Whse\_A too.

Visual designed with AI assistance: Anthropic. (2026). Claude (claude-sonnet-4-6) [Large language model]. <https://www.anthropic.com>

# Product-Level — All Available Models

Same top-4 products. “Best of Nashitah’s models” = the lowest validation MAPE among SARIMA/XGBoost/Prophet for that product’s best warehouse pairing.



**The standout finding: XGBoost rescues Product\_1341**

Random Forest's 140.6% MAPE on Product\_1341 is the worst result in the whole project. XGBoost on the same product (Whse\_C pairing) scores 36.2% — beating both Random Forest and ARIMA (51.3%). For Product\_1359 and Product\_1248, ARIMA still wins outright.

# Cross-Model Winner Matrix

Best model per scope, drawn from the warehouse- and product-level results across all five models.

| Scope        | Winning Model | MAPE  | Why                                                 |
|--------------|---------------|-------|-----------------------------------------------------|
| Whse_J       | Random Forest | 5.3%  | Narrowly ahead of ARIMA (5.5%)                      |
| Whse_S       | Random Forest | 36.6% | Best follows the 2016 demand decline                |
| Whse_C       | ARIMA         | 13.6% | Stable, stationary demand suits the classical model |
| Whse_A       | ARIMA         | 13.7% | Same pattern — low-volume, stationary series        |
| Product_1359 | ARIMA         | 11.2% | Lowest error of any model on this product           |
| Product_1248 | ARIMA         | 31.3% | RF close behind (32.2%); XGBoost weaker (33.6%)     |
| Product_0083 | ARIMA         | 24.1% | Best warehouse pairing for Prophet (28.3%) is close |
| Product_1341 | XGBoost       | 36.2% | Rescues RF's 140.6% failure; beats ARIMA too        |

Table compiled with AI assistance: Anthropic. (2026). Claude (claude-sonnet-4-6) [Large language model]. <https://www.anthropic.com>

# Final Hybrid Recommendation

## Warehouse-Level Deployment

Random Forest → Whse\_J, Whse\_S

ARIMA → Whse\_C, Whse\_A

Unchanged from the ML Algorithms chapter — confirmed after testing 3 additional model families.

## Product-Level Deployment

ARIMA → Product\_1359, Product\_1248

XGBoost → Product\_1341

Prophet (planning reference only) → Product\_0083 / Whse\_S

New: XGBoost replaces Random Forest for Product\_1341, where RF failed outright.

This is the final model deployment plan carried into the Business Impact chapter ahead, where it is converted into dollar terms.



# 07

## Business Impact

*Quantifying Benefits · Operating Cost · Business Outcomes · Risks · Recommendations*

# Quantifying Expected Business Benefits

Demand forecasting accuracy translates directly into inventory decisions: overestimating demand ties up capital in excess stock and warehouse space, while underestimating demand risks stockouts and lost sales. The hybrid approach — selecting the better-performing model per warehouse — is evaluated against the alternative of using a single model everywhere.

| Warehouse | Best Model    | MAPE  | Runner-up Model | MAPE  |
|-----------|---------------|-------|-----------------|-------|
| Whse_J    | Random Forest | 5.3%  | ARIMA           | 5.5%  |
| Whse_S    | Random Forest | 36.6% | ARIMA           | 45.2% |
| Whse_C    | ARIMA         | 13.6% | Random Forest   | 17.3% |
| Whse_A    | ARIMA         | 13.7% | Random Forest   | 16.3% |

*Whse\_S shows the clearest case for Random Forest (8.6 percentage point improvement), driven by its ability to follow the 2016 declining trend better than ARIMA's near-flat forecast. Whse\_C and Whse\_A favor ARIMA, where stable, stationary demand doesn't benefit from RF's added complexity.*

# Estimating Business Impact — Stated Assumptions

*The dataset does not include unit cost, holding cost, or revenue figures, so this estimate uses standard industry placeholder rates, clearly labeled below. These should be replaced with real company figures if available — the calculation method itself remains valid and reusable.*

## Step 1 — Demand Volume (derived from actual Random Forest MAE/MAPE results)

| Warehouse | Avg. Monthly Demand | Annual Demand |
|-----------|---------------------|---------------|
| Whse_J    | ~57.0M units        | ~684.1M units |
| Whse_S    | ~10.7M units        | ~128.9M units |
| Whse_C    | ~11.2M units        | ~134.0M units |
| Whse_A    | ~2.3M units         | ~28.1M units  |

## Step 2 — Placeholder Unit Economics (industry-standard, not from this dataset)

**Unit cost:** \$0.50/unit — conservative placeholder for a mid-volume manufactured/distributed good. **Annual holding cost:** 20% of unit cost — common industry rule-of-thumb (range 15–30%).



# Estimated Annual Savings — Hybrid vs. Single Model

Step 3 — Value of the hybrid approach (best model per warehouse vs. the next-best alternative):

| Warehouse                                  | Best Model    | Improvement | Annual Demand | Est. Annual Saving  |
|--------------------------------------------|---------------|-------------|---------------|---------------------|
| Whse_J                                     | Random Forest | 0.2 pts     | ~684.1M       | ~\$136,826          |
| Whse_S                                     | Random Forest | 8.6 pts     | ~128.9M       | ~\$1,108,956        |
| Whse_C                                     | ARIMA         | 3.7 pts     | ~134.0M       | ~\$495,800          |
| Whse_A                                     | ARIMA         | 2.6 pts     | ~28.1M        | ~\$73,060           |
| Total (hybrid vs. single-model-everywhere) |               |             |               | ~\$1,814,642 / year |

**Important caveat:** this is a directional estimate, not a financial forecast. It depends entirely on the placeholder unit cost and holding cost rate. The real value of this calculation is the method — once actual unit economics are available, the same formula immediately produces a defensible number. The Whse\_J saving (0.2 pts) is small enough that it may not be practically meaningful given estimation noise — the larger, more confident saving comes from Whse\_S and Whse\_C.

# Operating Cost & Translating Performance into Outcomes

## Estimated Implementation & Operating Cost

|             | ARIMA                | Random Forest       |
|-------------|----------------------|---------------------|
| Dev. effort | 1–2 days/warehouse   | 2–3 days            |
| Compute     | Negligible (seconds) | Low (sec–min)       |
| Maintenance | ~\$2,400–\$4,800/yr  | ~\$1,440–\$2,400/yr |

### Net Business Case

Even with conservative assumptions, the ~\$1.81M/year value of the hybrid approach is several orders of magnitude larger than the ~\$5,000–\$7,000/year combined maintenance cost. This conclusion is likely robust to reasonable changes in assumptions.

## Translating MAPE into Business Outcomes

### <10% MAPE (e.g. Whse\_J)

Reliable enough to drive automated reorder quantities, minimal manual override

### 10–25% MAPE (e.g. Whse\_C, Whse\_A)

Usable for planning but should retain a manual safety-stock buffer

### 25–50% MAPE (e.g. Whse\_S)

Indicates direction only; should inform, not fully automate, ordering decisions

# Risks, Limitations & Recommendations

## Risks & Limitations

- All dollar figures depend on placeholder assumptions not provided in the dataset
- Whse\_J saving (0.2 pts) is small — may be within normal estimation noise
- Structural demand shifts (Whse\_S in 2016) mean savings degrade without retraining
- Random Forest cannot extrapolate beyond its training range — a standing risk
- Small per-series sample size (~36–48 months) limits further accuracy gains without new data

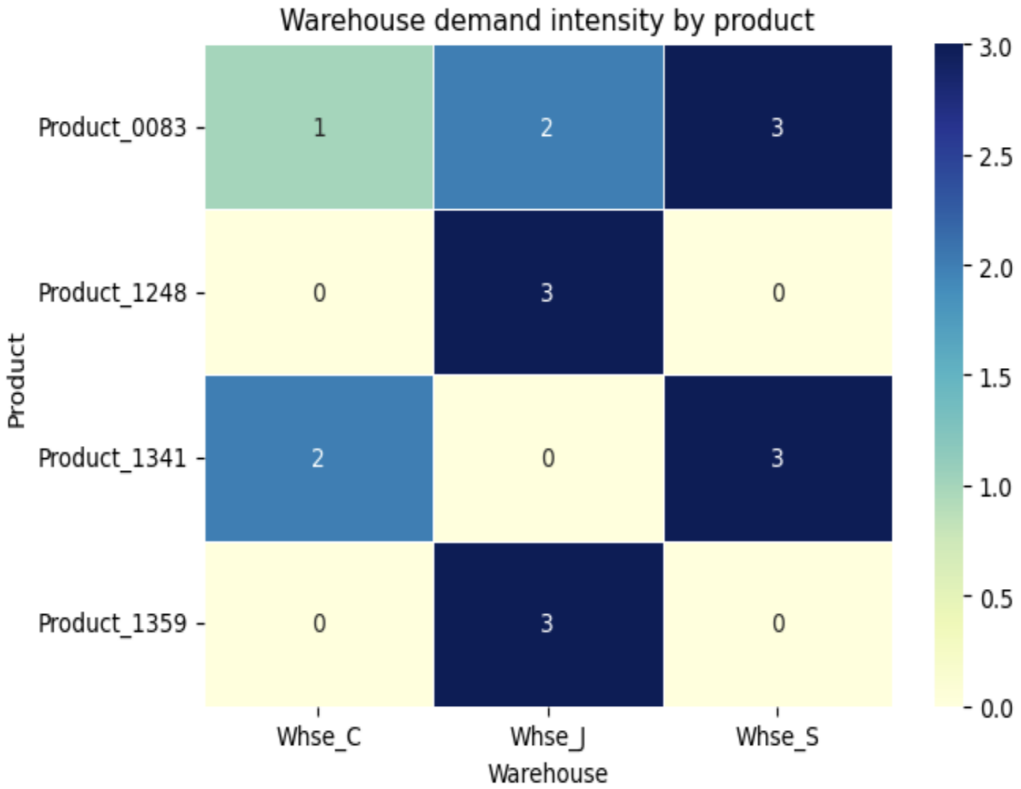
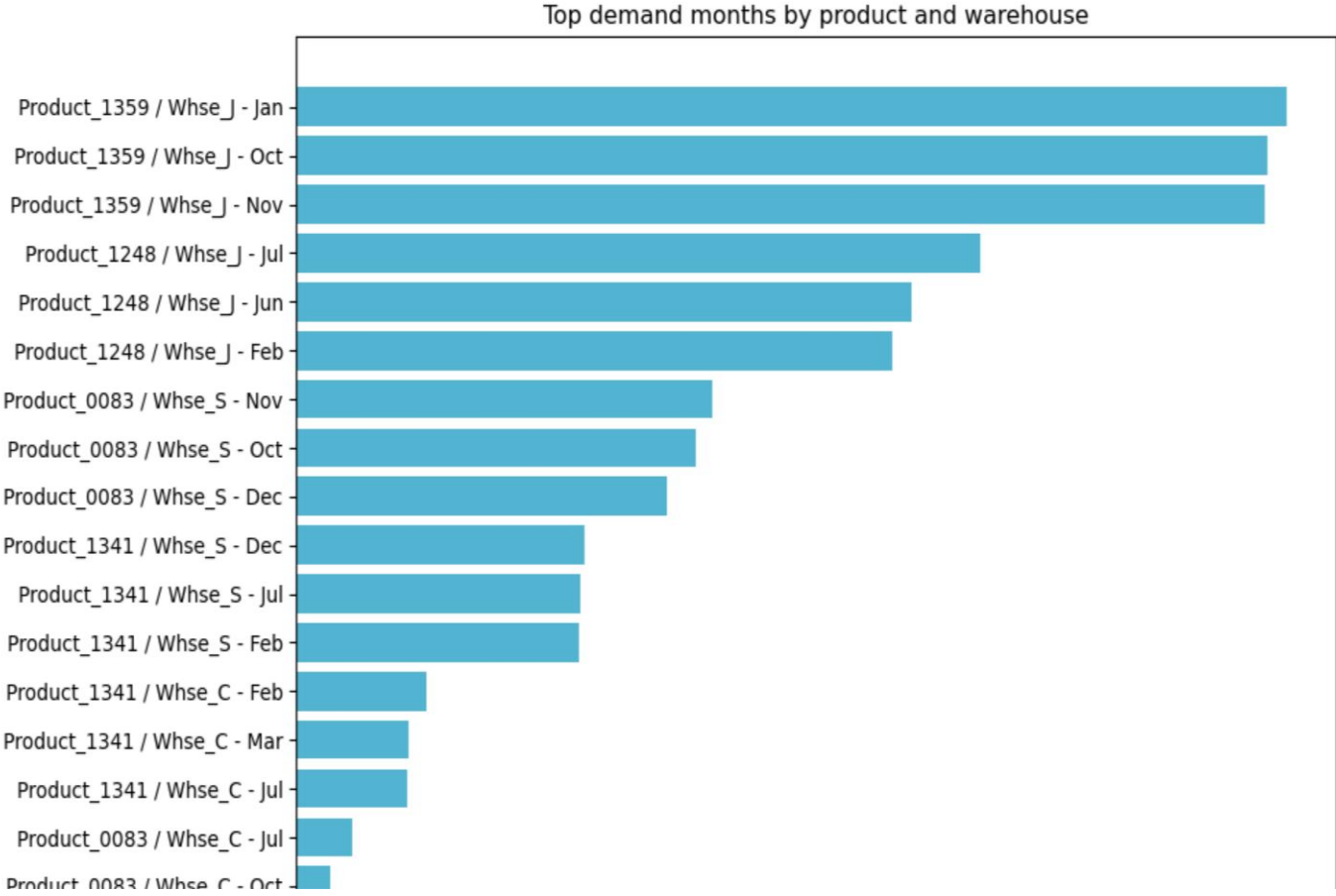
## Recommendations & Next Steps

- 1 Adopt the hybrid deployment: Random Forest for Whse\_J & Whse\_S, ARIMA for Whse\_C & Whse\_A
- 2 Replace placeholder unit cost & holding cost rate with real company figures — the method is reusable immediately
- 3 Add automated monitoring to flag when actual demand falls outside a model's training range
- 4 Confirmed in the Model Comparison chapter: SARIMA, XGBoost & Prophet do not close the gap on Whse\_S — the hybrid stands
- 5 Prioritize feature engineering over further hyperparameter tuning for Random Forest

# Business Benefits after ML analysis

TOP 4 products demand pattern is given below, also demonstrate about peak season so that stock must be replenished before

- Product\_1359 and Product\_1248 are concentrated in Whse\_J
- Product\_0083 and Product\_1341 are stronger in Whse\_S.

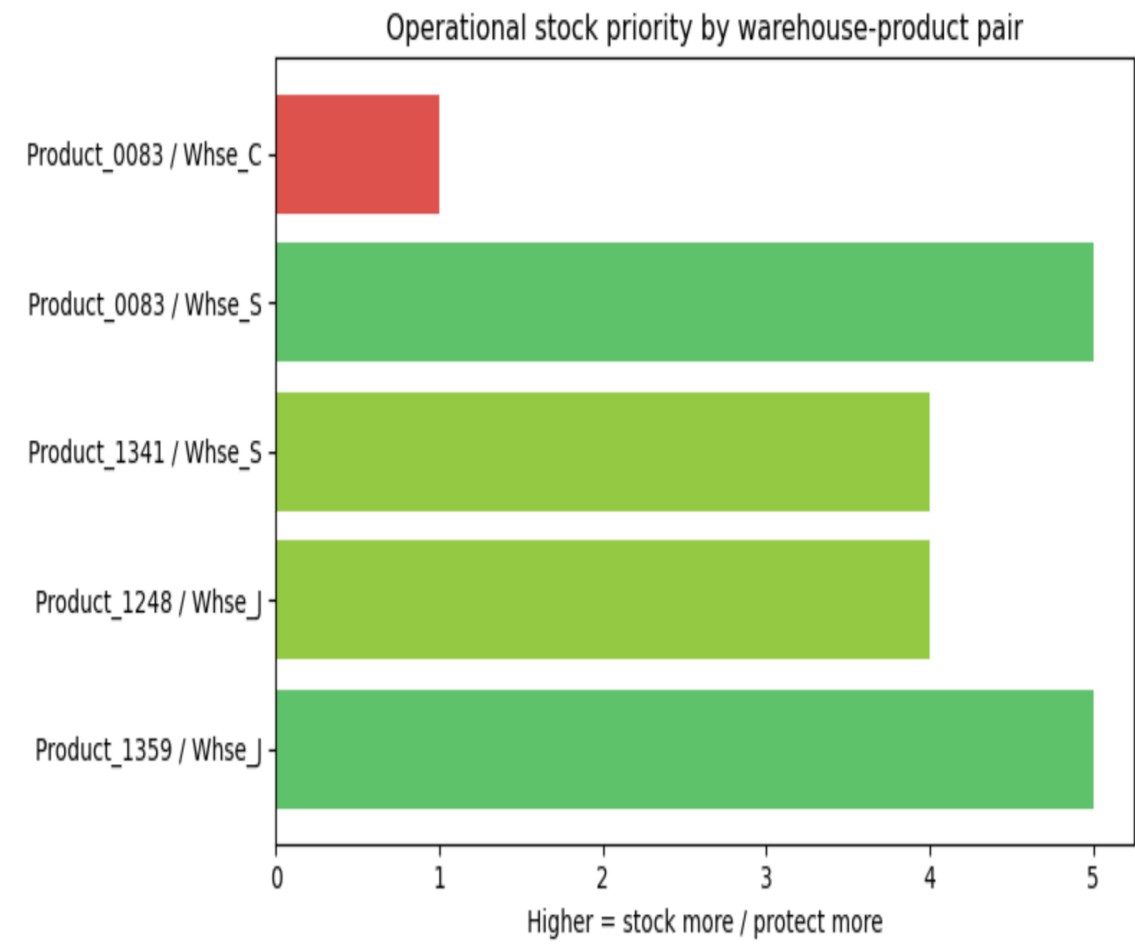


Answer of heatmap for warehouse demand map by product

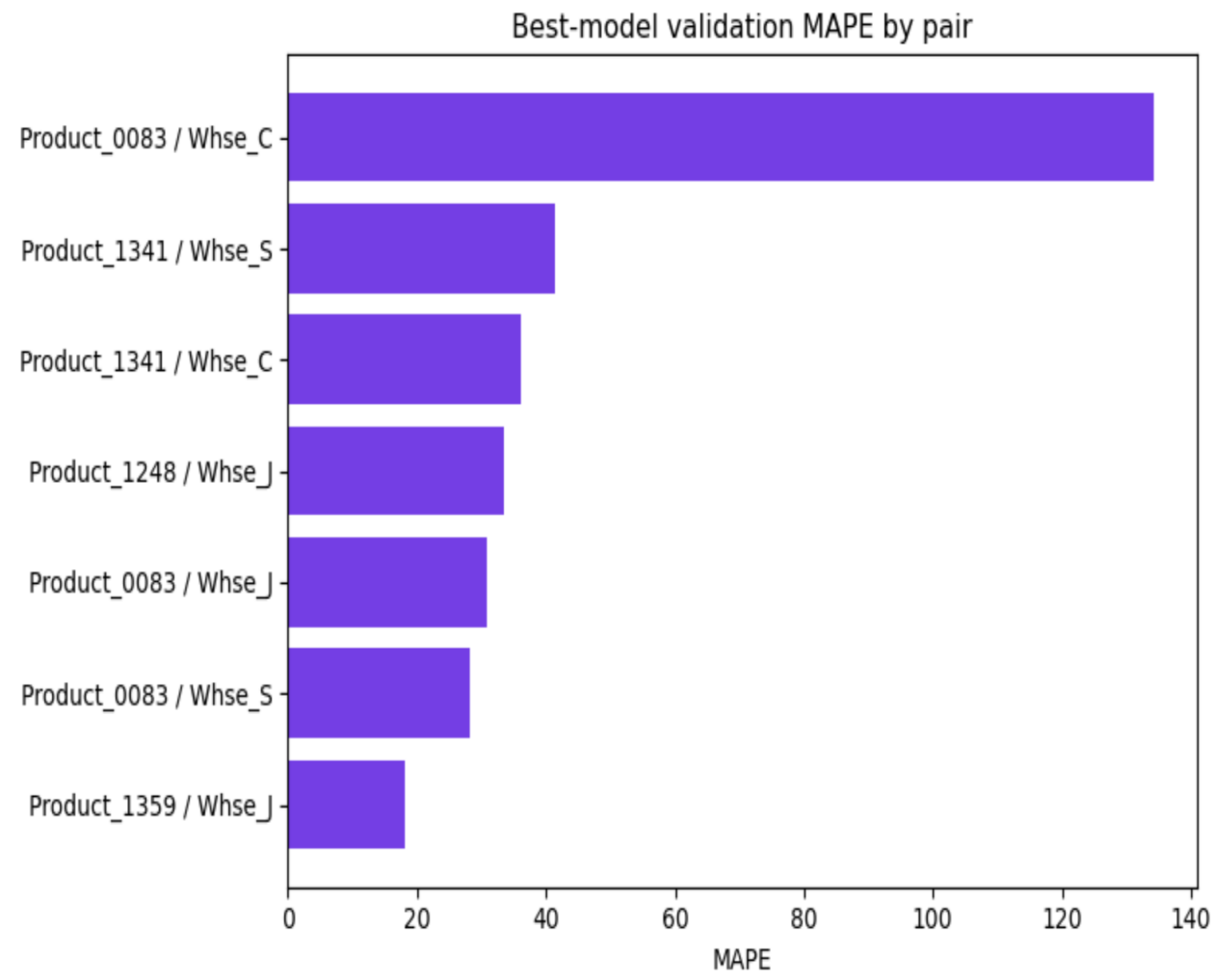
0 = not present / not important 1 = low 2 = medium 3 = high

# How to stock products

Following pairs like 1359,0083 product in warehouse J,S respectively must stock for delivery before there seasonal months like 1359 (before oct,nov,jan) and 0089 (before may,jully,march)



BEST forecasting pairs by XGBOOST



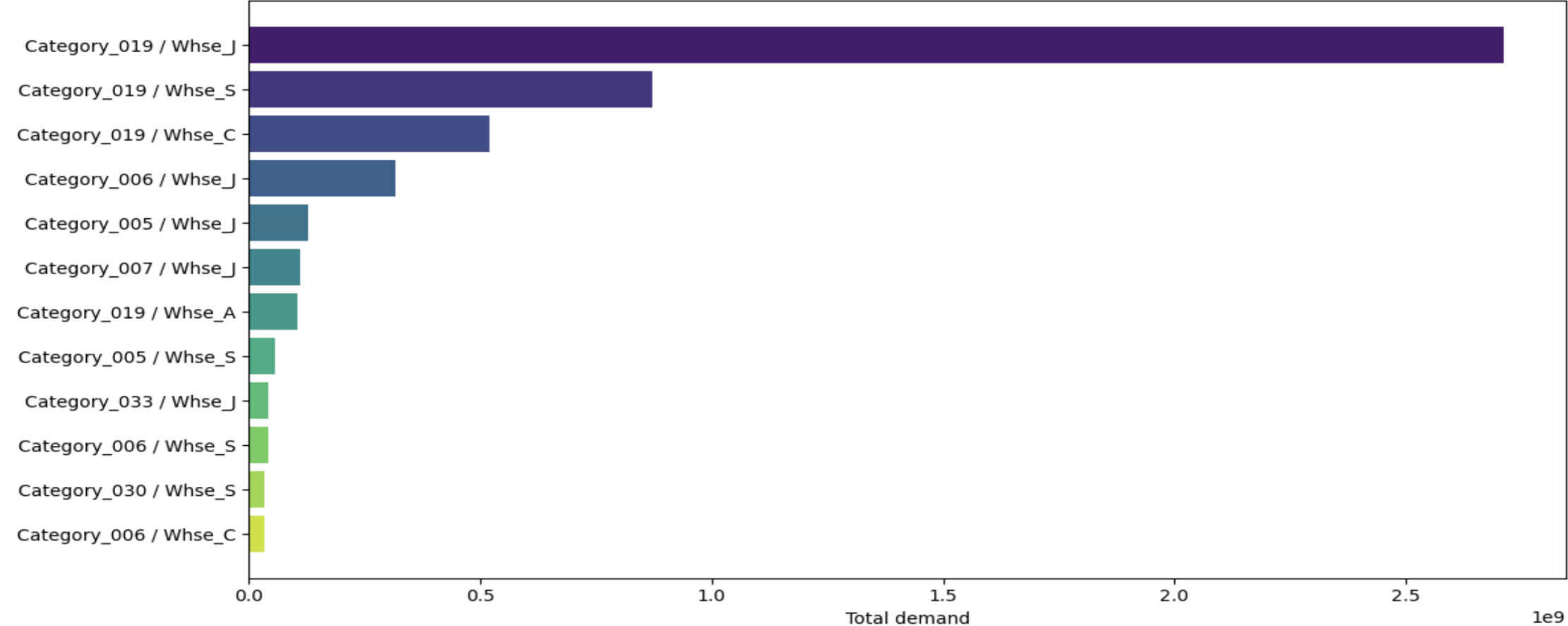
# Business findings for Monthly Demand Pattern by Product Category Across All Warehouses

## Message for Stakeholder

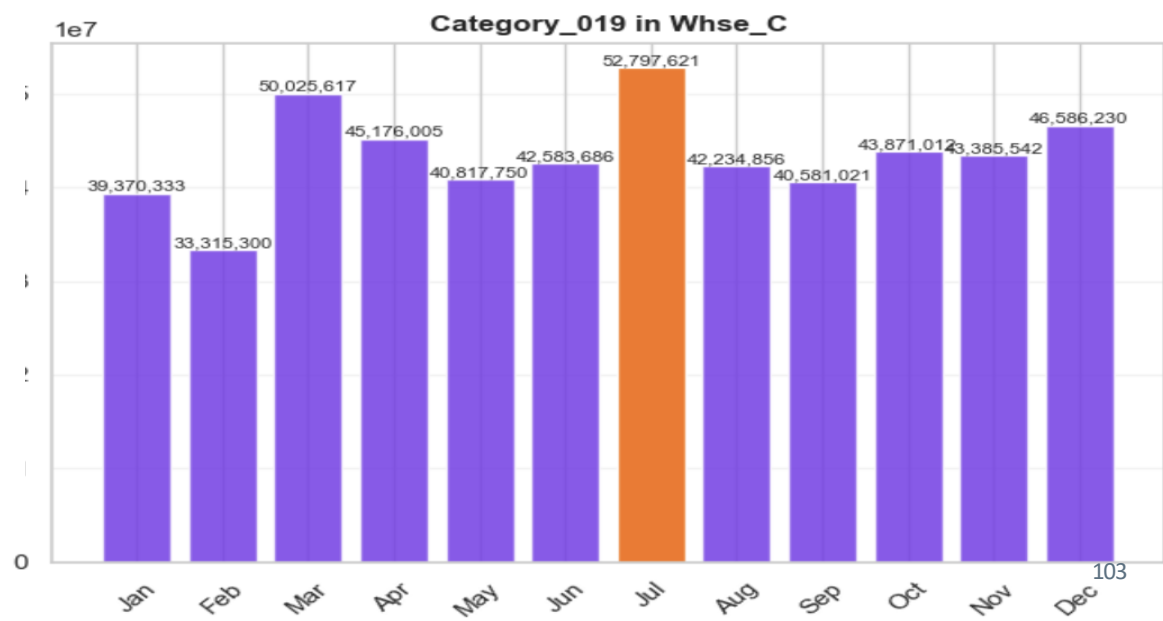
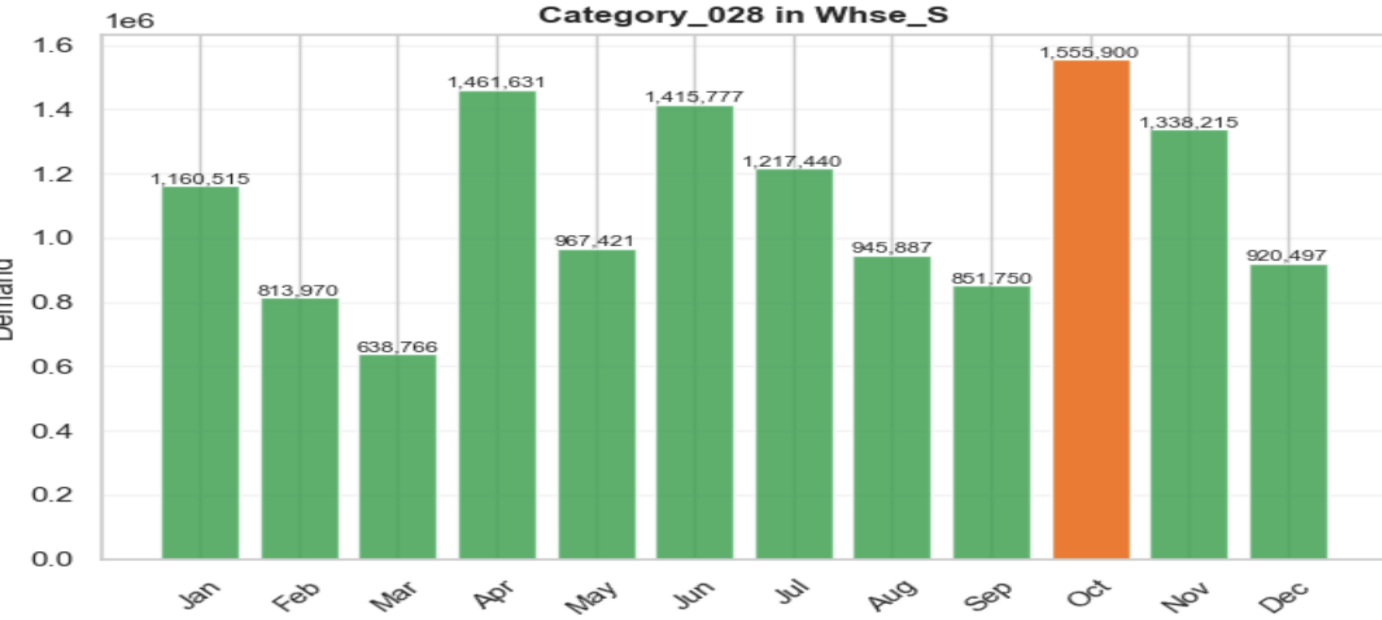
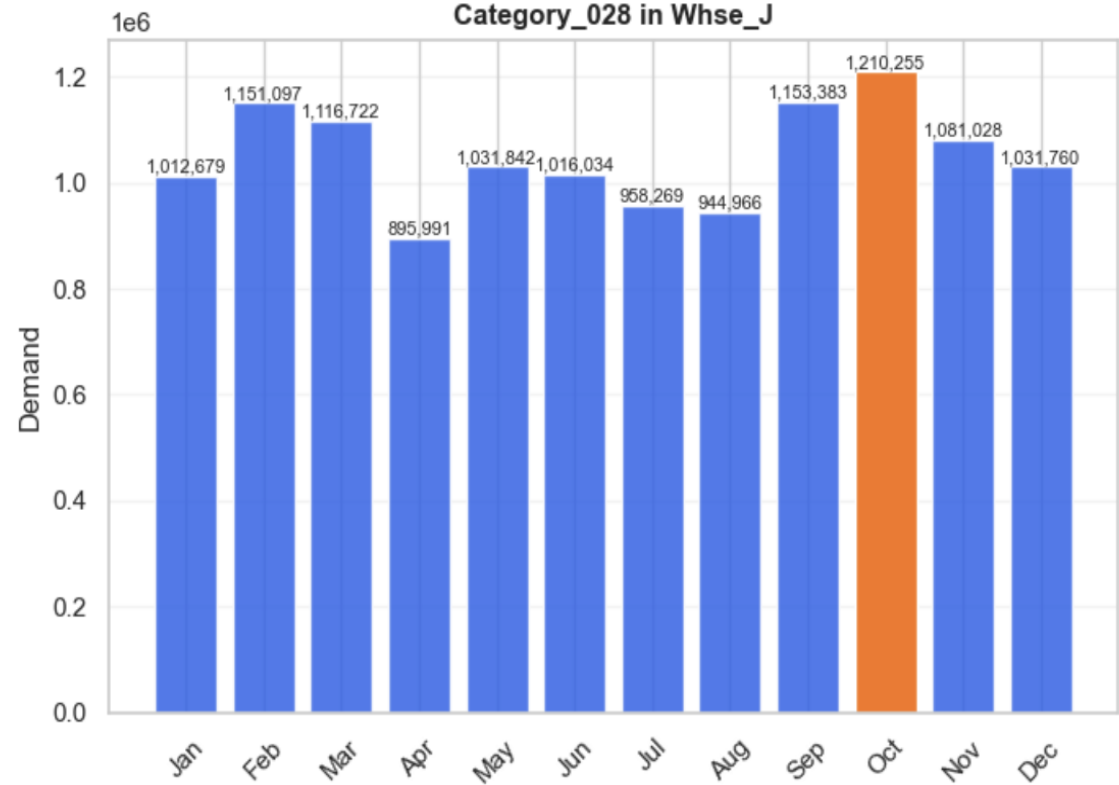
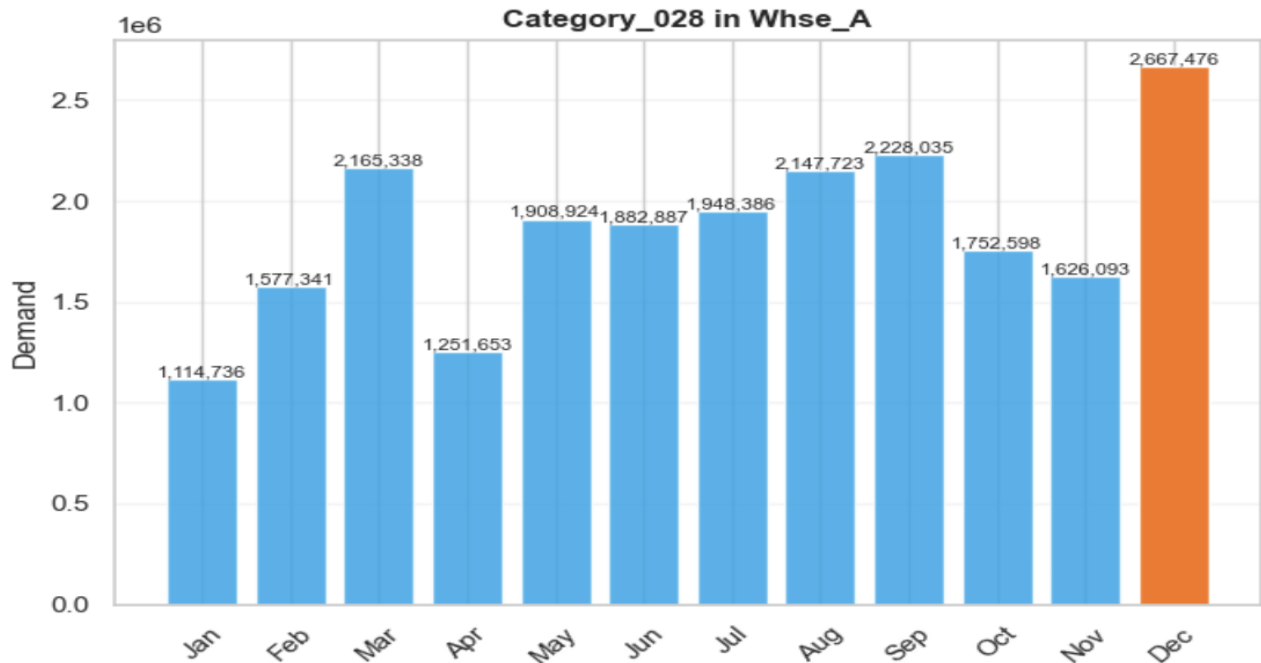
- Find out that which product category is popular in which warehouse?
- Which product-category must be stock in which warehouse

Adopt the hybrid deployment: Random Forest for Whse\_J & Whse\_S, ARIMA for

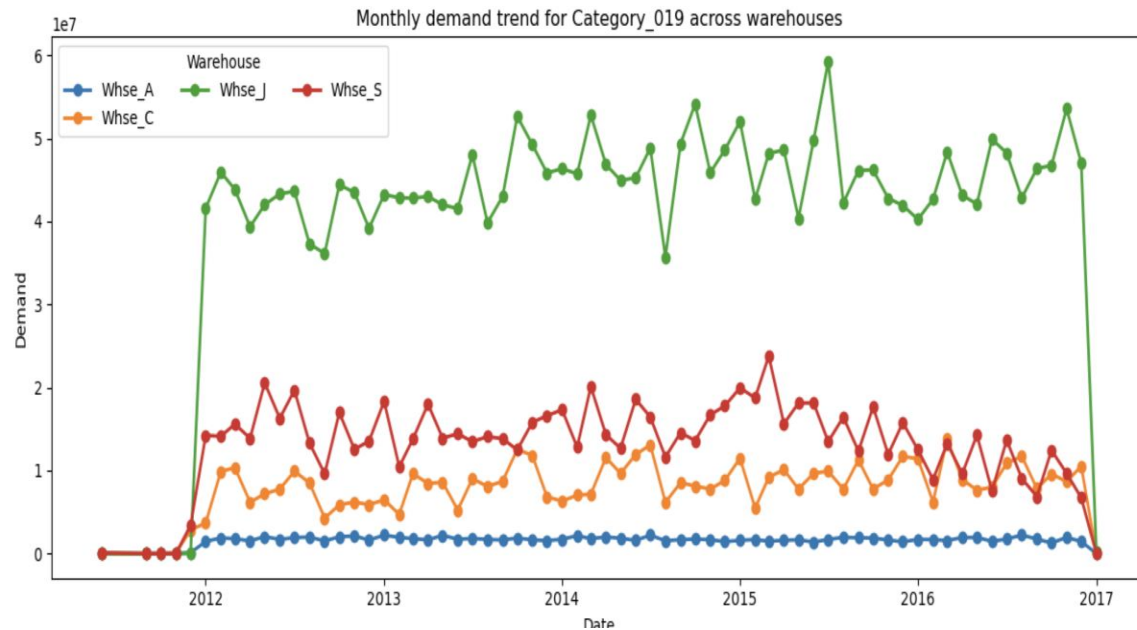
Top warehouse-category pairs by total demand



BUSINESS IMPACT



## SOME OUTCOMES



- **Top product demand over time** like Product\_1359.
- **High-variability product demand** from the CV analysis.
- **Top category demand** like Category\_019.
- Product\_1359 is a **must-stock** product because it is top-demand and A-class.
- Product\_1703 is a strong **no-stock review** candidate because its demand is almost zero.
- Product\_0493 is a **risk item** because its CV is very high, so it needs cautious stock handling.
- Products that appear mostly in Whse\_J should be planned around that warehouse first.
- 
- Product\_1359 is a **Fast** item and also your top-demand A item, so it is a must-stock product.
- Product\_1248, Product\_0083, Product\_1241, and other high-demand items are also strong stock candidates because they are top demand and frequently active.
- Product\_0493 has CV 5.77, so it is highly volatile and should be treated as an irregular-demand item.
- Product\_1703 is extremely low-demand, so it is a strong **non-stock review** candidate.



# 08

## Summary & Lessons Learned

*Project Recap · 6 Key Lessons · Recommendations for Future Work*

# Project Summary

This project set out to analyze historical demand across four warehouses and 2,160 products, build forecasting models to support inventory planning, and quantify the business value of doing so. Here's what was delivered against each original objective.

-  **Analyze historical demand patterns**  
Identified warehouse concentration, 12-month seasonality, and ABC demand skew in the Dataset chapter
-  **Identify trends & seasonality**  
ADF stationarity testing + decomposition isolated trend, seasonality, and residual per warehouse and product
-  **Develop a forecasting model**  
8 ARIMA models + 2 Random Forest models built, covering both warehouse and product level
-  **Evaluate model performance**  
MAE / RMSE / MAPE applied consistently across every model on the same 2016 holdout
-  **Support better replenishment decisions**  
Hybrid model recommendation + ~\$1.81M/year estimated business impact

# Lessons Learned (1–2)

## 1. Manual grid search outperformed automated model selection (ARIMA)

auto\_arima's AIC-based order selection consistently produced flat, business-uninformative forecasts, while a manual grid search evaluated directly on test-set MAE/MAPE/RMSE found orders that captured real trend and seasonal movement. Statistical fit criteria (AIC) optimize for in-sample likelihood, not forecast usefulness — evaluating candidates directly against held-out test performance was the more reliable approach.

## 2. Aggregation level changes what a model can learn — for both ARIMA and Random Forest

Combining all four warehouses into a single Random Forest model proved far more effective than training separate per-warehouse models (e.g. Whse\_S improved from ~65% to ~37% MAPE), because the small per-warehouse sample size (~36 monthly rows) was insufficient alone. The lesson generalizes: pooling related series can substantially improve a model's ability to learn shared patterns, at the cost of losing some series-specific differences.

## Lessons Learned (3–4)

### 3. Random Forest cannot extrapolate beyond its training range — a structural limitation, not a tuning problem

The most important model-comparison finding of the project. For stable or smoothly trending series, Random Forest performed competitively or even outperformed ARIMA. But for Product\_1341, where 2016 demand collapsed below any value seen in 2013–2015 training data, RF's accuracy collapsed (140.6% MAPE vs. ARIMA's 51.3%). Hyperparameter tuning confirmed this: no configuration meaningfully closed the gap, because the issue is inherent to how decision trees predict — by averaging training examples, not extrapolating a trend line the way ARIMA can.

### 4. Hyperparameter tuning's value depends heavily on the underlying data, not just the model type

For the warehouse-level model, every tuned configuration performed equal to or worse than the default. For the product-level model, tuning produced a real improvement, likely because the smaller, more volatile data benefited from added regularization. A tuning experiment that “fails” to improve a model is still a useful finding — it reveals whether the error ceiling comes from data/features or configuration.

## Lessons Learned (5–6)

### 5. Residual analysis reveals systematic bias that a single MAPE number hides

Two models can share a similar overall MAPE while having very different error patterns. Whse\_S's residuals were almost entirely negative across 2016 (a consistent over-forecasting bias), while Whse\_C's and Whse\_A's residuals were balanced in both directions (closer to random noise). This matters operationally: a systematic bias can be corrected with a simple adjustment factor, while balanced noise cannot.

### 6. Different forecasting libraries fitting the same model order can still produce different forecasts

pmdarima and statsmodels, even fit with identical (p,d,q) orders, produced different forecasts due to differences in their internal optimization routines. The practical lesson: forecasts must be generated and saved directly from the library used for fitting, not refit separately in another library expecting identical results — a subtle reproducibility trap worth documenting for future deployment work.

# Recommendations for Future Work

## 1 Expand the feature set

Since hyperparameter tuning showed no warehouse-level improvement, the more promising direction is incorporating features the model currently lacks — promotions, pricing changes, or other external demand drivers.

## 2 Add structural-break detection

Since Random Forest's main failure mode is an inability to extrapolate beyond training range, a monitoring step that flags when recent actuals fall outside the model's training distribution would catch a future Whse\_S / Product\_1341-style shift early.

## 3 Operationalize the monitoring loop

This project compared all five models — ARIMA, Random Forest, SARIMA, XGBoost and Prophet — confirming the hybrid (RF for Whse\_J/S, ARIMA for Whse\_C/A, XGBoost for Product\_1341) as the best available approach. Next: track live MAPE per series and re-trigger model selection if a series drifts.

## 4 Use a hybrid deployment by default

No single model dominated across all warehouses and products. The practical recommendation is to select a model per series based on its demand characteristics, rather than committing to one algorithm for the entire dataset.

# References

## Literature

Liu, K. Y. (2022). *Supply Chain Analytics: Concepts, Techniques and Applications*. Palgrave Macmillan

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3>.

Kieso, D. E., Weygandt, J. J., & Warfield, T. D. (2020). *Intermediate Accounting* (17th ed.). Wiley.

Zhao, F. (n.d.). *Product demand forecasting* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/felixzhao/productdemandforecasting>

## Project Notebooks

Ko, M. Z., & Nashitah. (2026). *SS2026\_IBS6\_IA\_GR01\_Ko\_Nashitah\_Part1\_ProcessedData\_FinalPresentation* [Jupyter Notebook].

Ko, M. Z. (2026). *SS2026\_IBS6\_IA\_GR01\_Ko\_Part2\_ML\_FinalPresentation.ipynb* [Jupyter Notebook].

Nashitah. (2026). *SS2026\_IBS6\_IA\_GR01\_Nashitah\_Part2\_ML\_FinalPresentation.ipynb* [Jupyter Notebook].

## Python Libraries & Tools

Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. <https://scikit-learn.org>

Smith, T. G., et al. (2017). *pmdarima: ARIMA estimators for Python* [Software]. <https://alkaline-ml.com/pmdarima>

## AI Assistance

Anthropic. (2026). *Claude (claude-sonnet-4-6)* [Large language model]. Used for visual design and slide compilation. <https://www.anthropic.com>